Universidade Federal Fluminense

Instituto de Computação

Departamento de Ciência da Computação

Atharv Nuthi

Felipe Vahia Malliagros

Arquitetura PASE: autonomia supervisionada com agentes de IA no ciclo de vida de software, a partir de uma análise multidimensional do ecossistema

Niterói

2026

Atharv Nuthi
Felipe Vahia Malliagros

Arquitetura PASE: autonomia supervisionada com agentes de IA no ciclo de vida de software, a partir de uma análise multidimensional do ecossistema

Trabalho de Conclusão de Curso submetido ao Departamento de Ciência da Computação da Universidade Federal Fluminense como requisito parcial para a obtenção do título de Bacharel em Ciência da Computação.

Orientador: Prof. Troy Costa Kohwalter

Niterói
2026

Inserir aqui a ficha gerada a partir do Sistema de Geração Automática de Fichas Catalográficas,
disponível no endereço https://bibliotecas.uff.br/bee/fichacatalografica

Atharv Nuthi
Felipe Vahia Malliagros

Arquitetura PASE: autonomia supervisionada com agentes de IA no ciclo de vida de software, a partir de uma análise multidimensional do ecossistema

Trabalho de Conclusão de Curso submetido ao Departamento de Ciência da Computação da Universidade Federal Fluminense como requisito parcial para a obtenção do título de Bacharel em Ciência da Computação.

Aprovado em XX de XXX de XXXX:

Prof. Troy Costa Kohwalter
Orientador - UFF

Professor
Instituição

Professor
Instituição

Niterói
2026

[Opcional. A redigir pelos autores.]

Opcional. A redigir pelos autores.

Declaração de Considerações Éticas

Esta pesquisa não envolveu experimentos com seres humanos nem a coleta de dados pessoais ou sensíveis. O produto-alvo usado na validação, o FinTrack, opera exclusivamente com duas personas fictícias e com dados financeiros sintéticos, gerados para fins de demonstração. A prova de conceito foi executada apenas em ambiente local, sem exposição pública e sem integração com dados bancários reais, o que afasta o tratamento de dados pessoais nos termos da Lei Geral de Proteção de Dados (Lei nº 13.709/2018).

Os riscos considerados na concepção da arquitetura dizem respeito à segurança do software produzido e à responsabilidade pelas decisões tomadas por agentes de IA. Esses riscos foram mitigados por decisões de projeto descritas no texto: execução isolada do agente que escreve código, sem acesso de rede além do provedor do modelo; verificação automática obrigatória; aprovação humana registrada antes de qualquer integração e de qualquer publicação; recusa de conteúdo personalizado que viole contratos rígidos; e registro integral de cada decisão. As chaves de acesso aos provedores de modelos foram mantidas fora do repositório.

Declaração de Uso de IA Generativa

Ferramentas de IA generativa foram utilizadas neste trabalho em duas condições distintas, declaradas a seguir.

Como objeto de estudo, modelos de linguagem integram a própria prova de conceito descrita no Capítulo 5. São eles o *gpt-4.1-mini*, nas etapas de filtragem de demandas; o *gpt-5.3-codex*, na escrita de código; o *gpt-5.5*, na revisão independente das entregas; e o *text-embedding-3-small*, na busca semântica. Todos são acessados pela interface de programação da OpenAI, e suas saídas são registradas e auditáveis.

Como ferramenta de apoio aos autores, foi utilizado o assistente Claude Code, da Anthropic, com modelos da família Claude [confirmar versões utilizadas]. Ele apoiou o desenvolvimento do código da prova de conceito, a organização e a revisão do texto, a verificação das citações contra os documentos de referência e a formatação do manuscrito. [Confirmar e completar com outras ferramentas eventualmente utilizadas, como corretores gramaticais ou tradutores.] Todo o conteúdo produzido com esse apoio foi revisado, verificado e assumido pelos autores, que são responsáveis pelas ideias, pelas decisões de projeto, pela análise e pelas conclusões apresentadas.

Resumo

A inteligência artificial agêntica desloca o software do papel de ferramenta que apoia o trabalho humano para o de sistema que executa trabalho de forma autônoma, e a adoção dessa tecnologia avança mais rápido do que as estruturas capazes de torná-la confiável. Este trabalho parte dessa lacuna entre capacidade técnica e confiabilidade e propõe a Arquitetura PASE (*Personalized Adaptive Software Ecosystem*), uma arquitetura híbrida humano-IA que automatiza o ciclo de vida do software com agentes de IA sob supervisão humana. Os requisitos da arquitetura derivam de uma análise multidimensional do ecossistema de software, conduzida por revisão narrativa estruturada de 22 trabalhos sobre indústria, desenvolvimento, experiência do usuário, segurança e governança, e mercado de trabalho, que resultou em cinco requisitos explícitos. A PASE organiza-se em três camadas: a Camada de Tarefas, que capta demandas de usuários, da equipe e da própria IA e as filtra com memória do que já foi implementado e recusado; a Camada de Orquestração, que prioriza, modela dependências e posiciona a decisão humana sobre a tarefa, o plano, a entrega e a publicação; e a Camada de Execução, que produz código em ambiente isolado, verificado por máquina e revisado por um agente independente, e que confina a personalização da interface em tempo de execução a zonas com contratos verificáveis. Para validar a proposta, foi implementada uma prova de conceito, aplicada à evolução de um painel financeiro pessoal. Uma demanda em linguagem natural percorreu as três camadas e foi publicada na versão 1.0.0 do produto, com decisão humana em cada ponto crítico, e o agente de personalização adaptou a interface por usuário e recusou, no pior caso, três conteúdos maliciosos que contornaram a validação do servidor. A validação também expôs falhas reais: metade das entregas aprovadas antes da introdução da revisão independente foi revertida, e as estimativas de custo do planejamento erraram por mais de uma ordem de grandeza. Cada falha foi absorvida pela arquitetura na forma de um mecanismo verificável. Como contribuições, o trabalho oferece a análise multidimensional e seus requisitos, a especificação da arquitetura, a prova de conceito com suas evidências e um princípio de projeto que emergiu da validação: garantias que dependem apenas de instrução ao modelo não são garantias, e devem ser verificadas por código ou decididas por uma pessoa.

Palavras-chave: Inteligência Artificial Agêntica. Engenharia de Software. Autonomia Supervisionada. Sistemas Multiagente. Interfaces Adaptativas. Governança de IA.

Abstract

Agentic artificial intelligence is shifting software from a tool that supports human work to a system that performs work autonomously, and its adoption is advancing faster than the structures needed to make it trustworthy. This work addresses that gap between technical capability and trustworthiness and proposes the PASE Architecture (Personalized Adaptive Software Ecosystem), a hybrid human-AI architecture that automates the software life cycle with AI agents under human supervision. The architecture's requirements derive from a multidimensional analysis of the software ecosystem, conducted through a structured narrative review of 22 works on industry, development, user experience, security and governance, and the labor market, which yielded five explicit requirements. PASE is organized into three layers: the Task Layer, which captures demands from users, the team and the AI itself and filters them against a memory of what has already been implemented and rejected; the Orchestration Layer, which prioritizes tasks, models dependencies and places human decisions over the task, the plan, the delivery and the release; and the Execution Layer, which produces code in an isolated environment, verified by machine and reviewed by an independent agent, and which confines runtime interface personalization to zones with verifiable contracts. To validate the proposal, a proof of concept was implemented and applied to the evolution of a personal finance dashboard. A natural-language request went through the three layers and was released in version 1.0.0 of the product, with a human decision at every critical point, and the personalization agent adapted the interface per user and, in the worst case, rejected three malicious payloads that bypassed server-side validation. The validation also exposed real failures: half of the deliveries approved before the independent review was introduced were later reverted, and the cost estimates produced during planning were off by more than an order of magnitude. Each failure was absorbed by the architecture as a verifiable mechanism. The contributions of this work are the multidimensional analysis and its requirements, the architecture specification, the proof of concept and its evidence, and a design principle that emerged from the validation: guarantees that rely only on instructions to the model are not guarantees, and must be verified by code or decided by a person.

Keywords: Agentic Artificial Intelligence. Software Engineering. Supervised Autonomy. Multi-Agent Systems. Adaptive Interfaces. AI Governance.

Lista de ilustrações

Figura 1 – Criação de valor acumulado projetada impulsionada por IA 5
Figura 2 – Visão geral da Arquitetura PASE 20
Figura 3 – Camada de Tarefas da Arquitetura PASE 21
Figura 4 – Camadas de Orquestração e de Execução da Arquitetura PASE 25
Figura 5 – Ciclo de vida de uma tarefa na Arquitetura PASE 28
Figura 6 – Contrato rígido de uma Zona Dinâmica: payload válido e payload recusado 33
Figura 7 – Descartadas: propostas barradas pelo funil por duplicidade ou incoerência, com o motivo de cada descarte 38
Figura 8 – Dependências: grafo das tarefas, com nós coloridos por estado e arestas de dependência 38
Figura 9 – Sistema: fila de execução, memória vetorial, consumo acumulado, travas e *prompts* versionados 38
Figura 10 – Contexto: documentos do produto versionados no banco e as etapas do ciclo que leem cada um 39
Figura 11 – FinTrack antes da solicitação: interface apenas em tema claro, sem controle de tema 40
Figura 12 – Painel do PASE recebendo a solicitação: demanda em linguagem natural e tarefa candidata qualificada, aguardando aprovação ou recusa 41
Figura 13 – Documento de Planejamento Técnico: arquivos previstos, critérios de aceite (C1 a C5) e trilho do ciclo de vida da tarefa 41
Figura 14 – Execução registrada: diferenças de código, registro das verificações automáticas e transcrição integral da interação com o modelo 41
Figura 15 – Homologação: parecer da revisão independente por critério de aceite e decisão do gestor (aprovar, ajustar ou recusar) 42
Figura 16 – Tela Entrega: tarefa no ramo de integração, versão 1.0.0 validada e publicação no ramo principal sob aprovação humana 42
Figura 17 – FinTrack após a entrega: botão “Usar tema escuro” e paleta escura aplicada a todas as telas 43
Figura 18 – Simulador do painel: o uso intenso de relatórios pela persona Ana aciona a zona do painel inicial e o UPA gera um payload declarativo 44
Figura 19 – Personalização em execução: Ana vê o componente gerado pelo UPA, com o selo “personalizado”, enquanto Clara mantém o componente nativo 44
Figura 20 – Defesa em profundidade: payload com link externo injetado sem validação do servidor, recusado pelo cliente e registrado como violação 45

Lista de quadros

Quadro 1 – Lacunas por dimensão e requisitos derivados para uma arquitetura de software agêntico 13
Quadro 2 – Comparação entre os trabalhos relacionados e a Arquitetura PASE . . 17
Quadro 3 – Componentes validados na PoC e escopo remanescente 37
Quadro 4 – Roteiros de demonstração e as propriedades que verificam 39
Quadro 5 – Linha do tempo da tarefa nº 6, conforme os registros do sistema (8 de outubro de 2026) 43
Quadro 6 – Ajustes motivados pela validação: do previsto ao construído 48
Quadro 7 – Prompts da prova de conceito e componentes correspondentes 56

Lista de tabelas

Tabela 1 – Tarefas executadas pela PoC e seus desfechos 45
Tabela 2 – Métricas de validação calculadas sobre as tarefas do período 46

Lista de abreviaturas e siglas

API	*Application Programming Interface*
CHI	*Conference on Human Factors in Computing Systems*
CSV	*Comma-Separated Values*
DAG	*Directed Acyclic Graph* (grafo acíclico direcionado)
DORA	*DevOps Research and Assessment*
HULA	*Human-in-the-Loop LLM-based Agents*
IA	Inteligência Artificial
ICLR	*International Conference on Learning Representations*
JSON	*JavaScript Object Notation*
LLM	*Large Language Model* (modelo de linguagem de grande escala)
PASE	*Personalized Adaptive Software Ecosystem*
PoC	*Proof of Concept* (prova de conceito)
QP	Questão de Pesquisa
RAG	*Retrieval-Augmented Generation* (geração aumentada por recuperação)
SOP	*Standard Operating Procedure* (procedimento operacional padronizado)
UFF	Universidade Federal Fluminense
UPA	*User Personalization Agent*
XSS	*Cross-Site Scripting*

Sumário

1 INTRODUÇÃO 1
1.1 Contexto 1
1.2 Objetivos 2
1.3 Metodologia 2
1.4 Organização do Texto 3

2 SOFTWARE NA ERA DA IA AGÊNTICA: IMPACTOS E LACUNAS 4
2.1 Indústria e Aplicações 4
2.2 Desenvolvimento de Software 6
2.3 Usuário e Experiência 8
2.4 Segurança, Ética e Governança 9
2.5 Mercado de Trabalho e Competências 10
2.6 Síntese: Lacunas e Requisitos para uma Arquitetura 12

3 TRABALHOS RELACIONADOS 14
3.1 Introdução 14
3.2 Supervisão Humana: HULA 14
3.3 Automação do Ciclo de Vida: MetaGPT 15
3.4 Originação de Demandas: LLM-Cure 15
3.5 Personalização em Tempo de Execução: Interfaces Generativas e Maleáveis 16
3.6 Considerações Finais 17

4 PROPOSTA: A ARQUITETURA PASE 18
4.1 Introdução 18
4.2 Visão Geral e Princípios de Projeto 18
4.3 Camada de Tarefas (Task Layer) 20
4.3.1 Formação de Requisitos 20
4.3.2 Middleware de Normalização 22
4.3.3 Avaliador de Zonas Dinâmicas 22
4.3.4 Motor de Filtragem de Tarefas 23
4.3.5 Módulo de Reavaliação Cíclica 24
4.4 Camada de Orquestração (Orchestration Layer) 24
4.4.1 Priorização e Decisão Humana 25
4.4.2 Planejamento, Dependências e Despacho 26
4.4.3 Tratamento de Falhas e Trava de Custo 26

4.4.4 Homologação e Entrega . . . 27
4.4.5 Ciclo de Vida da Tarefa . . . 27
4.4.6 Ciclo de Auditoria e Saneamento Recorrente . . . 28
4.5 Camada de Execução (Execution Layer) . . . **28**
4.5.1 Executor de Tarefas: Fase de Planejamento Técnico . . . 29
4.5.2 Executor de Tarefas: Fase de Execução e Verificação . . . 29
4.5.3 Revisão Independente e Retorno de Dados . . . 30
4.5.4 User Personalization Agent e Zonas Dinâmicas . . . 31
4.5.5 Governança das Zonas Dinâmicas . . . 32
4.5.6 Interoperabilidade entre Zonas . . . 33
4.6 Considerações Finais . . . **33**

5 IMPLEMENTAÇÃO E VALIDAÇÃO . . . **35**
5.1 Introdução . . . **35**
5.2 Implementação da Prova de Conceito . . . **35**
5.2.1 Produto-alvo e Arquitetura Técnica . . . 35
5.2.2 Correspondência entre a Arquitetura e a Implementação . . . 36
5.2.3 Painel de Controle . . . 36
5.3 Material e Métodos . . . **37**
5.4 Fluxo 1: da Demanda ao Código em Produção . . . **40**
5.5 Fluxo 2: Personalização em Tempo de Execução . . . **43**
5.6 Resultados Obtidos . . . **45**
5.7 Discussão . . . **46**
5.8 Considerações Finais . . . **47**

6 CONCLUSÃO . . . **49**
6.1 Contribuições . . . **49**
6.2 Limitações . . . **50**
6.3 Trabalhos Futuros . . . **51**

REFERÊNCIAS . . . **52**

APÊNDICES **55**

APÊNDICE A – PROMPTS VERSIONADOS DA PROVA DE CONCEITO . . . **56**

APÊNDICE B – CONTRATO CANÔNICO DAS ZONAS DINÂMICAS . . . **57**

APÊNDICE C – REPRODUÇÃO DA PROVA DE CONCEITO . . . 59

ANEXOS 60

1 Introdução

1.1 Contexto

O ecossistema de software, que sustenta grande parte da economia digital, atravessa uma transformação de natureza diferente das anteriores. A computação em nuvem mudou onde o software executa e como é cobrado. A IA agêntica muda o que o software faz. Modelos de linguagem de grande escala e, mais recentemente, sistemas capazes de planejar, decidir e agir com crescente independência deslocam o software do papel de ferramenta que apoia o trabalho humano para o de sistema que executa trabalho em nome das pessoas (SAROYA; MACNEILL, 2026).

A engenharia de software está no centro dessa mudança. A escrita de código, historicamente manual, evolui para uma prática de orquestração de agentes que escrevem código, avaliada e dirigida por pessoas (ANTHROPIC, 2026). A adoção já é ampla: 90% dos profissionais de tecnologia ouvidos pelo relatório DORA de 2025 usam IA no trabalho (GOOGLE CLOUD, 2025). Os resultados, contudo, ainda não acompanham a adoção. O mesmo relatório mostra que a IA amplifica tanto as forças quanto as disfunções das organizações, que 30% dos profissionais confiam pouco ou nada no código gerado e que a adoção continua associada a maior instabilidade na entrega de software.

A tensão se repete em outras dimensões do ecossistema. Interfaces que se adaptam a cada usuário melhoram a usabilidade percebida, mas podem desorientar e corroer a confiança quando mudam sem explicação (ARORA et al., 2025). Agentes que agem sozinhos ampliam a superfície de ataque: a maioria dos agentes avaliados em um estudo recente mostrou-se vulnerável à manipulação por instruções maliciosas (LUPINACCI *et al.*, 2025 *apud* CHHABRA *et al.*, 2026). No trabalho, o uso desatento da IA reduz a formação das competências necessárias para supervisioná-la (SHEN; TAMKIN, 2026).

O problema que motiva este trabalho, portanto, não é a falta de capacidade técnica. É a falta de uma arquitetura que transforme essa capacidade em algo confiável. As organizações dispõem de agentes, mas não de um plano de controle que os coordene, que preserve a decisão humana nos pontos em que a ação gera consequência, que torne o custo observável e que registre cada decisão de forma auditável (ABSHER; KUPRIYANOVA, 2025; SPARK, 2026). Sem essa estrutura, a autonomia crescente dos agentes tende a gerar fragmentação, retrabalho e risco em vez de valor.

Este trabalho responde a esse problema com a Arquitetura PASE (*Personalized Adaptive Software Ecosystem*), uma arquitetura híbrida humano-IA que estrutura o ciclo de vida completo de uma funcionalidade, da originação da ideia à sua entrega e personalização

em produção, sob um regime de autonomia supervisionada. Na PASE, os agentes de IA propõem, decompõem, executam e verificam, enquanto as pessoas governam as decisões críticas e respondem por elas.

1.2 Objetivos

O objetivo geral deste trabalho é propor e validar a Arquitetura PASE, uma arquitetura híbrida humano-IA em três camadas, de Tarefas, de Orquestração e de Execução, que automatiza o ciclo de vida do software com agentes de IA sob supervisão humana. Os requisitos da arquitetura derivam de uma análise multidimensional do ecossistema de software, conduzida por revisão da literatura publicada entre 2017 e 2026 sobre indústria, desenvolvimento, experiência do usuário, segurança e governança, e mercado de trabalho.

Duas questões de pesquisa orientam o estudo. A primeira (QP1) consiste em investigar se a evolução da IA conduz o software a um ecossistema totalmente autônomo de agentes, que substitua progressivamente os modelos tradicionais, ou a sistemas híbridos humano-IA, que demandam novas arquiteturas, competências e formas de governança. A segunda (QP2) consiste em investigar como uma arquitetura de software pode equilibrar a autonomia crescente dos agentes de IA com a necessidade permanente de supervisão, controle e alinhamento humano, e se essa arquitetura é viável quando implementada.

Para alcançar o objetivo geral e responder a essas questões, foram definidos seis objetivos específicos. Os dois primeiros sustentam a análise: (i) analisar o impacto da IA agêntica em cinco dimensões do ecossistema de software e identificar as lacunas que separam a capacidade técnica de uma adoção confiável; e (ii) derivar dessas lacunas um conjunto de requisitos para uma arquitetura de software agêntico. O terceiro sustenta a proposta: (iii) especificar a Arquitetura PASE, seus princípios de projeto, suas camadas e seus pontos de decisão humana. Os três últimos sustentam a validação: (iv) implementar uma prova de conceito da arquitetura e aplicá-la à evolução de um produto real; (v) validá-la por meio de execuções reais, roteiros de demonstração, testes automatizados e métricas de supervisão; e (vi) discutir as limitações da proposta e os caminhos para sua evolução.

1.3 Metodologia

O trabalho foi conduzido em três etapas, que correspondem às três partes centrais do texto: uma revisão da literatura, a proposição de uma arquitetura e a validação dessa arquitetura por meio de uma prova de conceito. A primeira etapa consistiu em uma revisão narrativa estruturada da literatura, organizada pelas cinco dimensões analisadas no Capítulo 2. Foram incluídos trabalhos que tratam de modelos de linguagem ou de IA

agêntica em pelo menos uma dessas dimensões, publicados entre 2017 e 2026, em inglês ou português, com texto integral acessível e com método declarado, fosse um estudo empírico, uma revisão ou um levantamento com amostra descrita. Foram excluídos textos opinativos sem dados, materiais promocionais sem metodologia e versões duplicadas de um mesmo trabalho. A busca considerou bases de publicações científicas de computação, como ACM Digital Library, IEEE Xplore, ScienceDirect e SpringerLink, além do repositório arXiv e de publicações oficiais de organizações de pesquisa e de mercado.

O corpus resultante reúne 22 trabalhos. Desses, nove são artigos publicados em periódicos ou eventos, oito deles em veículos com revisão por pares, e cinco são versões preliminares disponíveis no arXiv. Os demais são um relatório técnico de pesquisa industrial e sete relatórios de indústria e de mercado, como o relatório DORA, produzido a partir de uma pesquisa com 4.867 profissionais. A inclusão de versões preliminares e de relatórios de indústria foi deliberada. Os fenômenos analisados, como agentes de programação em produção e arquiteturas multiagente, intensificaram-se em 2025 e 2026, em um ritmo que os periódicos ainda não acompanham. Para equilibrar atualidade e rigor, a revisão sistemática de Hou et al. (2024), que cobre 395 artigos publicados desde 2017, foi usada como referência para o panorama acadêmico, e os relatórios de indústria foram empregados apenas quando declaram amostra ou método. Os trabalhos relacionados do Capítulo 3 foram selecionados separadamente, entre propostas publicadas em eventos e periódicos revisados por pares.

A segunda etapa foi uma pesquisa propositiva. As lacunas identificadas na revisão foram convertidas em requisitos (Seção 2.6), e os requisitos, em princípios de projeto e componentes da arquitetura (Capítulo 4). A terceira etapa foi a validação por estudo de caso: uma prova de conceito da arquitetura foi implementada e aplicada à evolução de um produto real, um painel financeiro pessoal. Seu comportamento foi observado em execuções com modelos de linguagem reais, e as evidências foram coletadas dos registros do próprio sistema e do histórico do repositório (Capítulo 5).

1.4 Organização do Texto

O restante deste trabalho está organizado em cinco capítulos. O Capítulo 2 analisa o software na era da IA agêntica em cinco dimensões, identifica as lacunas de cada uma e as converte nos requisitos que orientam a proposta. O Capítulo 3 apresenta os trabalhos relacionados e delimita a contribuição deste estudo em relação a eles. O Capítulo 4 apresenta a Arquitetura PASE, seus princípios de projeto e suas três camadas. O Capítulo 5 descreve a implementação da prova de conceito e sua validação em dois fluxos de ponta a ponta, com os resultados obtidos e sua discussão. Por fim, o Capítulo 6 apresenta as conclusões, as contribuições, as limitações do estudo e os trabalhos futuros.

2 Software na Era da IA Agêntica: Impactos e Lacunas

A Arquitetura PASE não nasce de uma preferência técnica, mas de uma leitura do momento que o ecossistema de software atravessa. Este capítulo apresenta essa leitura. Ele percorre as cinco dimensões nas quais a IA agêntica produz efeitos mais visíveis, a saber, (i) indústria e aplicações, (ii) desenvolvimento de software, (iii) usuário e experiência, (iv) segurança, ética e governança e (v) mercado de trabalho e competências. Em cada uma, o objetivo não é catalogar tendências, mas identificar a lacuna que a separa de uma adoção confiável. A Seção 2.6 reúne essas lacunas e as converte nos requisitos que orientam a proposta do Capítulo 4.

2.1 Indústria e Aplicações

A história do software empresarial é uma sucessão de modelos que, a cada ciclo, redefiniram não apenas a tecnologia, mas a natureza do que se vende. Saroya e MacNeill (2026) descrevem essa trajetória em três estágios: o software como produto, vendido por licença e instalado localmente; o software como serviço, entregue pela nuvem e cobrado por usuário; e o software como trabalhador, no qual o valor passa a ser medido pelas tarefas executadas. A mudança em curso é de outra natureza. Segundo os autores, a IA agêntica transforma o software empresarial de ferramenta que apoia o trabalho em sistema que executa o trabalho em nome das pessoas.

A escala econômica dessa transição ajuda a explicar a urgência com que as organizações a perseguem. O investimento corporativo em IA triplicou em um único ano e chegou a US$ 37 bilhões em 2025, e a camada de aplicação concentrou mais da metade desses recursos (MENLO VENTURES, 2025 *apud* SAROYA; MACNEILL, 2026). Projeções da Bain & Company estimam que entre US$ 5 e 7 trilhões em valor acumulado devem fluir para a camada de software e aplicações até 2030. Trata-se de uma expansão superior a quatro vezes em relação aos níveis atuais (BAIN & COMPANY, 2024 *apud* SAROYA; MACNEILL, 2026), conforme a Figura 1.

A adoção individual acompanha esse movimento. O relatório do programa DORA (*DevOps Research and Assessment*) de 2025, baseado em 4.867 profissionais de tecnologia de mais de cem países, mostrou que 90% dos respondentes já usam IA no trabalho, alta de 14,1% sobre o ano anterior (GOOGLE CLOUD, 2025). Os casos empresariais confirmam que a capacidade técnica já não é hipotética. Na Rakuten, um agente de programação implementou uma alteração em uma biblioteca de 12,5 milhões de linhas em sete horas de

Figura 1 – Criação de valor acumulado projetada impulsionada por IA



Fonte: Saroya e MacNeill (2026), a partir de projeções da Bain & Company (2024).

trabalho autônomo, com 99,9% de precisão numérica em relação ao método de referência. Na Zapier, a adoção de IA chegou a 89% da organização, com mais de oitocentos agentes em uso interno (ANTHROPIC, 2026).

Os casos mais maduros, porém, têm um traço em comum que vai além da autonomia: a estrutura. A plataforma Uma, da Duck Creek Technologies, coordena agentes especializados que executam as etapas da subscrição de seguros dentro de um ambiente estruturado e controlado, por onde passam mais de US$ 60 bilhões em prêmios por ano (SAROYA; MACNEILL, 2026). Na Fountain, a orquestração hierárquica de agentes acelerou a triagem de candidatos em 50% e a integração de novos funcionários em 40% (ANTHROPIC, 2026). Saroya e MacNeill (2026) sintetizam o padrão: a IA gera recomendações probabilísticas, mas o valor desproporcional surge quando as ações finais são executadas de forma determinística, dentro de regras, aprovações e trilhas de auditoria definidas.

O contraste entre a capacidade demonstrada e o resultado capturado define a primeira lacuna. Apenas cerca de 15% das organizações usam a IA para redesenhar de fato como o trabalho é feito, e ganhos de produtividade na escala da tarefa não têm se convertido em impacto na escala da empresa (WORLD ECONOMIC FORUM; ACCENTURE, 2026). Na pesquisa da SPARK (2026), 54% dos líderes não conseguem atribuir os custos do processamento autônomo a valor de negócio, e apenas 14% dos diretores financeiros relatam impacto claro e mensurável. O mesmo levantamento aponta a fragmentação entre funções como a principal barreira para 58% das organizações, e nomeia o risco que dela decorre: a proliferação descontrolada de agentes em silos isolados, o *Agent Sprawl*.

O mercado ainda não estabilizou sequer a definição do que é um agente. A Information Services Group observa a prática de *agent-washing*, em que soluções convencionais são rebatizadas como agênticas, e registra que apenas 25% das soluções atuais permitem que agentes operem de forma independente, enquanto 45% os posicionam como conselheiros de decisores humanos (ABSHER; KUPRIYANOVA, 2025). Os autores concluem que o papel da supervisão humana permanece vagamente definido e que os provedores mais avançados usam a orquestração como camada estruturada de controle, que impõe governança em vez

de ser ela mesma autônoma.

A dimensão industrial revela, portanto, uma convergência inédita de capacidade técnica, escala de investimento e pressão competitiva, mas também a ausência de uma arquitetura que transforme essa capacidade em valor governável. O que falta não é mais um agente, e sim um plano de controle que coordene agentes, preserve a decisão humana nos pontos críticos, torne o custo observável e deixe rastro de cada ação.

2.2 Desenvolvimento de Software

A engenharia de software sempre buscou organizar a complexidade de um processo essencialmente humano. Dos modelos sequenciais às metodologias ágeis, cada paradigma reorganizou as etapas do ciclo de vida, mas manteve a pessoa como agente central de decisão, criação e responsabilidade. Os modelos de linguagem de grande escala (*Large Language Models*, LLMs) rompem esse padrão porque passam a produzir artefatos em praticamente todas as fases do ciclo, dos requisitos à manutenção (BURTE, 2025; ZHANG et al., 2026).

A pesquisa acadêmica reflete essa expansão, mas de forma muito desigual. A revisão sistemática de Hou et al. (2024), com 395 artigos publicados entre 2017 e 2024, mostra que o desenvolvimento de código concentra 56,65% dos estudos, seguido da manutenção, com 22,71%, e da garantia de qualidade, com 15,14%. A engenharia de requisitos responde por apenas 3,9%, o projeto de software por 0,92% e a gestão de software por 0,69%. O levantamento de Zhang et al. (2026), com 926 estudos classificados em 112 tarefas, confirma o desequilíbrio e conclui que a aplicação de LLMs a requisitos, projeto e gestão continua relativamente pouco explorada.

Esse desequilíbrio tem consequências práticas. A fase em que a IA mais avança, a escrita de código, depende de decisões tomadas antes dela, sobre o que construir e por quê, justamente onde a IA menos foi estudada. Hou et al. (2024) observam ainda que, quando a intenção é ambígua, os modelos tendem a produzir código sintaticamente correto e funcionalmente errado, e que apenas seis dos 395 estudos analisados usaram dados industriais. A automação acelera a etapa mais visível do ciclo e deixa a qualidade das decisões que a antecedem por conta de processos que não acompanharam essa aceleração.

A qualidade estrutural do código gerado é a segunda frente de preocupação. Kravchuk-Kirilyuk, Graciolli e Amin (2025) mostram, em experimentos controlados, que um assistente de programação violou o encapsulamento ao acessar membros privados de um módulo em vez de usar sua interface pública, criando dependências ocultas entre módulos. Os autores descrevem o fenômeno como mimetismo arquitetural sem aderência aos princípios subjacentes: o código parece modular, mas não é. Soluções que funcionam na superfície tornam a base de código frágil, e o custo dessa fragilidade aparece apenas

mais tarde, na manutenção.

A dificuldade se intensifica quando a tarefa é real. No SWE-bench, que reúne 2.294 problemas extraídos de repositórios públicos em Python, o melhor modelo avaliado resolveu apenas 1,96% dos casos, em grande parte porque os modelos têm dificuldade em localizar o trecho de código que precisa ser alterado em meio a milhares de arquivos (JIMENEZ et al., 2024). Ainda que os modelos tenham evoluído desde então, o achado permanece instrutivo: resolver um problema real exige saber onde mexer, e esse é um problema de planejamento antes de ser um problema de geração.

Os dados de uso confirmam que a percepção de ganho nem sempre corresponde à medição. Mais de 80% dos respondentes do relatório DORA acreditam que a IA aumentou sua produtividade, mas 30% relatam pouca ou nenhuma confiança no código gerado (GOOGLE CLOUD, 2025). O mesmo relatório cita um estudo em que desenvolvedores 19% mais lentos com o uso de ferramentas de IA acreditavam ter ficado 20% mais eficientes. No plano organizacional, a adoção de IA passou a melhorar o volume de entregas em 2025, mas continua associada a maior instabilidade na entrega de software, o que sugere que os sistemas subjacentes ainda não evoluíram para gerenciar com segurança o desenvolvimento acelerado por IA.

A tese central do relatório DORA é que a IA atua como amplificadora: ela magnifica as forças das organizações de alto desempenho e as disfunções das demais (GOOGLE CLOUD, 2025). Entre quinze capacidades testadas, sete mostraram efeito moderador sobre os ganhos, entre elas um ecossistema de dados saudável, práticas sólidas de controle de versão, trabalho em pequenos lotes e foco no usuário. Na ausência dessas capacidades, os ganhos podem ser completamente neutralizados. A fricção, observa o relatório, não desaparece; ela se desloca do trabalho manual para a decisão e a verificação.

A orquestração de múltiplos agentes, apontada como a próxima fronteira do desenvolvimento (ANTHROPIC, 2026), não elimina esse problema e pode agravá-lo. He, Treude e Lo (2025) relatam que um sistema multiagente não conseguiu desenvolver de forma autônoma um jogo simples de Tetris e argumentam que a decomposição de requisitos e a gestão de interdependências se tornam mais difíceis à medida que a complexidade cresce. Kim et al. (2026), em uma avaliação controlada de arquiteturas de coordenação, encontraram efeitos que variam de ganhos de 80,8% a perdas de 70% conforme a tarefa. Em problemas de engenharia de software do SWE-bench Verified, ainda que em uma amostra reduzida, todas as variantes multiagente ficaram abaixo do agente único. Os autores mostram também que agentes que trabalham de forma independente, sem verificação, amplificam erros em até 17,2 vezes, enquanto uma coordenação centralizada, que valida os resultados antes de agregá-los, reduz essa amplificação para 4,4 vezes.

O desenvolvimento de software vive, portanto, uma transformação genuína, mas incompleta. A velocidade de geração cresceu, enquanto a qualidade das decisões anteriores

ao código, a integridade estrutural do que é gerado e a verificação do que é entregue não acompanharam o mesmo ritmo. A lacuna não está na capacidade de escrever código, mas na ausência de um processo que submeta essa capacidade a planejamento explícito, verificação determinística e decisão humana informada.

2.3 Usuário e Experiência

A história da interação humano-computador pode ser lida como uma sequência de interfaces cada vez mais próximas da intenção humana. Mukti e Trisilia (2025), em uma análise bibliométrica de vinte mil documentos publicados entre 1980 e 2024, identificam três fases da personalização. A primeira é uma fase técnica, baseada em regras e filtragem colaborativa, e a segunda, uma fase de escalabilidade, marcada pelo aprendizado profundo. A terceira é uma fase ética, iniciada em 2020, em que a IA generativa permite adaptação em tempo real e a pesquisa passa a priorizar transparência, privacidade e controle do usuário.

A fronteira atual é a das interfaces que deixam de ser artefatos fixos. Arora et al. (2025) descrevem sistemas que se autoevoluem, expandindo o próprio conjunto de funcionalidades em tempo de execução a partir da intenção imediata do usuário. Em um estudo com 72 participantes usando um protótipo bancário, a versão adaptativa obteve 84,38 pontos na Escala de Usabilidade do Sistema, contra 53,96 da versão determinística, uma diferença grande e estatisticamente significativa. Os autores atribuem o ganho à redução do esforço de navegação: em vez de o usuário aprender o modelo mental do sistema, o sistema passa a aprender o do usuário.

O mesmo trabalho, contudo, enumera os riscos dessa abordagem. Uma interface que muda constantemente pode desorientar, porque as pessoas dependem de memória espacial e de leiautes consistentes. Mudanças frequentes aumentam a carga cognitiva, modelos generativos tendem a produzir elementos visualmente inconsistentes e os métodos tradicionais de teste não cobrem o espaço de configurações possíveis (ARORA et al., 2025). Como resposta, os autores recomendam preservar âncoras visuais imutáveis, como a navegação global, e manter os dados transacionais intocados, independentemente de como a apresentação evolui.

A adaptação também precisa ser percebida para ter valor. Jamali et al. (2025), ao comparar três plataformas educacionais com 23 participantes, constataram que as funcionalidades adaptativas foram percebidas como sutis e pouco impactantes, e que a interatividade central das plataformas pesou mais na experiência do que a IA. Os autores identificam um compromisso entre eficiência e engajamento, mediado pela visibilidade das adaptações, e recomendam torná-las explícitas, anunciando ao usuário quando e por que o sistema mudou. A personalização invisível, concluem, não gera valor percebido.

A confiança é a variável que conecta essas evidências. Arora et al. (2025) afirmam que usuários desconfiam de interfaces que mudam sem explicação e que é crítico informar por que cada mudança acontece. Bieniek, Rahouti e Verma (2024) apontam a natureza de caixa-preta dos modelos como fonte de desconfiança, especialmente em domínios sensíveis como finanças. Na revisão de Tan et al. (2025), com 127 estudos sobre plataformas adaptativas, as visualizações que tornavam transparentes as recomendações aumentaram a confiança dos usuários. Mukti e Trisilia (2025) propõem mecanismos como um botão que explique cada recomendação.

A personalização tem ainda um custo de privacidade e de interpretação. A precisão da adaptação depende da coleta de dados comportamentais que podem causar desconforto, e a literatura sobre o tema dá pouca atenção à privacidade (TAN et al., 2025). Arora et al. (2025) defendem que o usuário seja informado sobre o uso de seus dados e possa recusar ou controlar o nível de personalização. Tan et al. (2025) alertam também que um sinal comportamental isolado é ambíguo: uma fixação longa do olhar pode indicar dificuldade, e não engajamento, o que exige combinar várias características antes de adaptar.

A dimensão do usuário mostra que a personalização em tempo real já entrega ganhos mensuráveis de usabilidade, mas expõe uma tensão entre adaptação e previsibilidade, entre conveniência e transparência. A lacuna não é técnica, mas de governança da interface: falta uma forma de personalizar que seja delimitada, explícita para o usuário, reversível ao comportamento padrão e incapaz de introduzir no produto código que não foi revisado.

2.4 Segurança, Ética e Governança

A segurança de software evoluiu por camadas sucessivas de proteção contra acesso indevido a dados e sistemas. Na IA agêntica, surge uma descontinuidade qualitativa: o adversário não precisa roubar informações, mas pode manipular o próprio processo de decisão do sistema. O relatório anual da CrowdStrike (2026) abre com a constatação de que o mundo opera na era agêntica, e de que o adversário também. Em 2025, os ataques conduzidos por adversários habilitados por IA cresceram 89% em relação ao ano anterior, e a IA acelerou o *phishing* e o reconhecimento automatizado, encurtando o tempo entre o acesso inicial e o impacto.

O mesmo relatório registra que os próprios sistemas de IA passaram a integrar a superfície de ataque das organizações, incluindo modelos, dados de treinamento e agentes (CROWDSTRIKE, 2026). Em um caso relatado, adversários embutiram instruções ocultas em um *e-mail* de *phishing* para confundir a triagem automatizada por IA, embora a técnica ainda não tenha demonstrado eficácia consistente em escala. A vulnerabilidade dos agentes é estrutural: um estudo citado por Chhabra et al. (2026) encontrou 94,4% dos agentes avaliados vulneráveis à injeção de instruções (*prompt injection*), 83,3% a *backdoors*

baseados em recuperação e 100% à exploração da confiança entre agentes (LUPINACCI *et al.*, 2025 *apud* CHHABRA *et al.*, 2026).

O código gerado por IA é outro vetor de risco. Modelos treinados com dados de baixa qualidade podem gerar código inseguro, e ataques que contaminam os dados de treinamento podem induzir vulnerabilidades sutis no código produzido (HOU et al., 2024). Quando agentes executam ações, e não apenas recomendam, a defesa recomendada pela literatura combina isolamento da execução, menor privilégio e modelos de proteção que examinam as entradas antes que alcancem o sistema. Chhabra et al. (2026) ressaltam, porém, que o isolamento é mais eficaz quando combinado a defesas complementares, e que confirmações humanas também podem ser alvo de engenharia social ou de aprovações desatentas.

A governança de sistemas autônomos é o ponto mais frágil desse quadro. A Information Services Group resume a mudança em uma frase: o que é fundamentalmente novo é a necessidade de governar a ação, já que cada agente capaz de agir sozinho introduz um risco operacional que os modelos atuais de governança ainda não sabem administrar (ABSHER; KUPRIYANOVA, 2025). A rastreabilidade também se torna mais difícil. Uma vez processada por um modelo, a informação é convertida em representações latentes distribuídas, que eliminam os identificadores capazes de ligar uma ação à sua origem e tornam inviável o rastreamento convencional de fluxo de informação (CHHABRA et al., 2026).

A dimensão ética acrescenta o problema da responsabilização. Abou Ali e Dornaika (2025) observam que a pergunta sobre quem responde por um dano é especialmente aguda em sistemas agênticos baseados em redes neurais, cuja natureza difusa e estocástica desafia marcos jurídicos construídos sobre causalidade direta e intenção. Os autores identificam um padrão por paradigma: sistemas simbólicos predominam em domínios críticos de segurança, como a saúde, enquanto sistemas neurais prevalecem em ambientes adaptativos e ricos em dados, como as finanças. A conclusão é que o caminho mais produtivo é híbrido, com módulos simbólicos garantindo restrições e conformidade sobre decisões baseadas em dados.

A segurança, a ética e a governança de sistemas agênticos não podem ser tratadas como problemas exclusivamente técnicos, nem resolvidas por uma única camada de proteção. A lacuna é a ausência de arquiteturas que tornem a ação dos agentes contida, verificável e atribuível: execução isolada, contratos que limitem o que pode ser produzido, decisões humanas nos pontos em que a ação gera consequência e registros que permitam reconstruir quem decidiu o quê.

2.5 Mercado de Trabalho e Competências

A pergunta mais frequente sobre IA e trabalho, a da substituição, recebe da literatura analisada uma resposta mais nuançada do que o debate público sugere. Os desenvolvedores usam IA em cerca de 60% do seu trabalho, mas relatam conseguir delegar por completo apenas entre 0% e 20% das tarefas, e cerca de 27% do trabalho assistido por IA corresponde a tarefas que não teriam sido feitas de outra forma (ANTHROPIC, 2026). O papel do engenheiro se expande em vez de desaparecer: ser engenheiro passa a significar orquestrar agentes que escrevem código, avaliar sua produção, dar direção estratégica e garantir que o sistema resolva o problema certo.

O deslocamento atinge sobretudo a estrutura dos papéis. Na pesquisa da SPARK (2026), cerca de metade das organizações aponta a capacitação de funcionários para gerenciar, auditar e colaborar com agentes como sua principal adaptação da força de trabalho, e mais de 46% estão abandonando escopos fixos de cargos em favor de uma evolução contínua de papéis. O Fórum Econômico Mundial registra o surgimento de funções como dono de produto de IA, arquiteto de fluxos de trabalho e curador de modelos, e descreve a média gerência migrando da supervisão de tarefas para a orquestração de fluxos híbridos entre pessoas e IA (WORLD ECONOMIC FORUM; ACCENTURE, 2026).

Essas mudanças vêm acompanhadas de tensão. A SPARK (2026) identifica a ansiedade de deslocamento como tema persistente e descreve as respostas das organizações mais bem-sucedidas. Elas reinvestem parte das economias da automação na capacitação das pessoas, dão aos funcionários visibilidade sobre os agentes implantados e reservam cerca de 20% do tempo para explorações que a IA não consegue simular. O relatório do Fórum acrescenta que o planejamento da força de trabalho deve considerar a capacidade combinada de pessoas e agentes, incluindo a nova carga de revisão e de tratamento de exceções que os próprios agentes criam (WORLD ECONOMIC FORUM; ACCENTURE, 2026).

O risco mais sutil está na formação de competências. Em um experimento controlado sobre o aprendizado de uma nova biblioteca de programação, o grupo que usou assistência de IA obteve, em média, 17% menos pontos na avaliação de conhecimento do que o grupo de controle (SHEN; TAMKIN, 2026). A diferença dependeu do modo de uso: participantes que delegaram a geração de código ao assistente ficaram entre 24% e 39% na avaliação, enquanto os que usaram a IA para perguntas conceituais ou pediram explicações junto ao código ficaram entre 65% e 86%. Os autores alertam que profissionais podem não desenvolver as habilidades necessárias para validar e depurar o código escrito pela IA.

O relatório DORA chega a uma conclusão convergente por outro caminho. Os padrões de uso mais comuns da IA estão entregando ganhos expressivos de produtividade e, ao mesmo tempo, bloqueando o desenvolvimento de habilidades da maioria dos

desenvolvedores, porque a automação reduz as oportunidades de aprendizado que antes cabiam aos iniciantes (GOOGLE CLOUD, 2025). Há aqui um paradoxo de longo prazo: a supervisão humana, que todas as dimensões anteriores apontam como indispensável, depende de pessoas capazes de julgar o que a IA produz, e essa capacidade é exatamente o que o uso desatento da IA corrói.

A dimensão do trabalho mostra que o futuro mais provável não é o da substituição em massa, mas o da redefinição de papéis em torno de orquestração, verificação e julgamento. A lacuna é a de estruturas que façam da supervisão humana um trabalho qualificado e informado, e não uma formalidade de aprovação, e que preservem, no próprio fluxo de trabalho, o engajamento cognitivo de quem decide.

2.6 Síntese: Lacunas e Requisitos para uma Arquitetura

Tomadas em conjunto, as cinco dimensões revelam um padrão consistente. Em todas elas, a capacidade técnica já demonstrada contrasta com a ausência de estruturas que a tornem confiável. A indústria tem agentes, mas não tem um plano de controle. O desenvolvimento tem geração de código, mas não tem verificação proporcional à velocidade. A experiência do usuário tem adaptação, mas não tem governança da interface. A segurança tem ameaças novas, mas não tem contenção arquitetural da ação. O trabalho tem novos papéis, mas não tem a estrutura que faça da supervisão um ato qualificado.

Essas lacunas não são independentes. A proliferação de agentes sem coordenação amplia a superfície de ataque. A verificação insuficiente do código transfere ao revisor humano uma carga que ele nem sempre tem competência ou contexto para sustentar. A personalização sem transparência corrói a confiança que a adoção exige. Por isso, tratá-las isoladamente, com uma ferramenta para cada sintoma, tende a reproduzir a mesma fragmentação que as originou.

Este trabalho interpreta esse conjunto como a evidência de uma lacuna central: a falta de uma arquitetura que equilibre a autonomia técnica dos sistemas de IA com supervisão humana efetiva, transparência e rastreabilidade, ao longo de todo o ciclo de vida do software. Dessa interpretação derivam cinco requisitos para uma arquitetura que pretenda preencher essa lacuna, apresentados no Quadro 1 e retomados no Capítulo 4.

Os cinco requisitos convergem para uma escolha de projeto. A via apontada pela evidência não é a do ecossistema totalmente autônomo, nem a da IA reduzida a assistente, mas a da autonomia supervisionada: ampla capacidade de ação dos agentes, subordinada a pontos de decisão humana e a garantias verificáveis por código. É essa escolha que a Arquitetura PASE, apresentada no Capítulo 4, procura materializar, depois de o Capítulo 3 situá-la em relação aos trabalhos que mais se aproximam dela.

Quadro 1 – Lacunas por dimensão e requisitos derivados para uma arquitetura de software agêntico

Dimensão	**Lacuna identificada**	**Requisito derivado**
Indústria e Aplicações	Capacidade técnica sem plano de controle; proliferação de agentes, custo pouco observável e supervisão vagamente definida	R1: coordenar agentes sob um plano de controle único, com custo observável e registro de cada decisão
Desenvolvimento de Software	Avanço concentrado na geração de código; requisitos e projeto pouco atendidos; código plausível, mas estruturalmente frágil	R2: cobrir o ciclo inteiro, da originação da demanda à entrega, e verificar o efeito da mudança, e não apenas o código
Usuário e Experiência	Adaptação com ganho de usabilidade, mas com risco de imprevisibilidade, opacidade e perda de controle	R3: personalizar em áreas delimitadas, de forma explícita, reversível ao padrão e sem gerar código não revisado
Segurança, Ética e Governança	Agentes vulneráveis a manipulação; ação difícil de rastrear e de atribuir	R4: isolar a execução, limitar a produção por contratos verificáveis e reservar a pessoas as decisões que geram consequência
Mercado de Trabalho e Competências	Papéis redefinidos em torno de orquestração e verificação; risco de erosão da competência de quem supervisiona	R5: tornar a supervisão humana informada, com evidências e contexto suficientes para que a decisão seja um julgamento, e não uma formalidade

Fonte: elaborado pelos autores (2026).

3 Trabalhos Relacionados

3.1 Introdução

Este capítulo apresenta os trabalhos que mais se aproximam da proposta. A seleção partiu das quatro frentes que compõem a Arquitetura PASE. As duas primeiras são (i) a supervisão humana no fluxo de desenvolvimento com agentes e (ii) a automação do ciclo de vida por sistemas multiagente. As duas seguintes são (iii) a originação de demandas a partir de fontes não estruturadas e (iv) a personalização de interfaces em tempo de execução. Para cada frente, foi escolhido o trabalho mais próximo da PASE entre os publicados em veículos revisados por pares de computação, com prioridade para propostas que descrevem um sistema implementado e avaliado, e não apenas uma visão. Para cada trabalho, descrevem-se sua proposta, os pontos de convergência com a PASE e as diferenças que delimitam a contribuição deste estudo. A Seção 3.6 consolida a comparação no Quadro 2.

3.2 Supervisão Humana: HULA

O trabalho mais próximo da PASE em termos de supervisão humana é o HULA (*Human-in-the-Loop LLM-based Agents*), proposto por Takerngsaksiri et al. (2025) e implantado em produção no Jira, da Atlassian. O *framework* organiza-se em três agentes que cooperam para resolver uma *issue*. O *AI Planner Agent* localiza os arquivos afetados e gera um plano de codificação, e o *AI Coding Agent* implementa o plano. O *Human Agent* é o engenheiro que revisa, refina e aprova o plano e o código antes da abertura de um *pull request*. Em dois meses de uso, os profissionais acionaram o HULA em 663 *issues*; foram gerados 527 planos, 433 deles aprovados, e 56 *pull requests* acabaram integrados.

Essa sequência corresponde ao trecho da PASE que vai da Camada de Orquestração à Camada de Execução, em especial ao Documento de Planejamento Técnico, exigido antes da escrita de código, e à homologação humana com autoridade final. Os próprios resultados do HULA reforçam a necessidade dessa supervisão: na pesquisa com 109 engenheiros, apenas 33% concordaram que o código gerado resolvia a *issue* sem intervenção humana, e 54% discordaram de que ele estivesse livre de defeitos (TAKERNGSAKSIRI et al., 2025). O HULA, contudo, parte de uma *issue* já formulada e priorizada por pessoas. Ele não trata a originação nem a filtragem de demandas, não modela interdependências entre tarefas, não guarda memória das rejeições e não atua sobre o software em tempo de execução. A PASE pode ser entendida como uma generalização do princípio do HULA, que desloca a supervisão humana de pontos isolados de revisão para um plano de controle

que acompanha a tarefa da captação à personalização em produção.

3.3 Automação do Ciclo de Vida: MetaGPT

Na amplitude do ciclo de vida coberto, destaca-se o MetaGPT (HONG et al., 2024), o *framework* multiagente que mais se aproxima da PASE. Nele, agentes assumem papéis de uma equipe de software (gerente de produto, arquiteto, gerente de projeto, engenheiro e engenheiro de qualidade) e seguem Procedimentos Operacionais Padronizados (*Standard Operating Procedures*, SOPs). Um requisito de uma linha é progressivamente transformado em documento de requisitos, projeto de sistema, lista de tarefas, código e testes, com artefatos estruturados trocados entre os papéis, em linha de montagem, para reduzir a propagação de alucinações. Os autores relatam 85,9% e 87,7% de acerto na primeira tentativa nos *benchmarks* HumanEval e MBPP, e um custo de revisão humana menor que o de abordagens anteriores baseadas em conversa entre agentes.

A decomposição de uma demanda em etapas com artefatos intermediários verificáveis e a separação entre quem planeja e quem executa são princípios compartilhados pela PASE, cuja Camada de Execução também produz um documento de planejamento antes da implementação. A diferença central está no papel da pessoa e na origem das demandas. O MetaGPT busca simular uma organização de software inteira com agentes e concentra a intervenção humana no requisito inicial. A PASE, por sua vez, mantém o gestor como responsável pela priorização, pela aprovação do plano e pela homologação, e trata a estratégia de coordenação entre agentes como detalhe interno do Executor. Essa escolha está em consonância com a evidência de que a eficácia de sistemas multiagente depende da adequação entre a estrutura de coordenação e a tarefa, e não do número de papéis envolvidos (KIM et al., 2026). Além disso, o MetaGPT não prevê isolamento obrigatório da execução, controle de custo, auditoria recorrente da base de código nem adaptação em tempo de execução.

3.4 Originação de Demandas: LLM-Cure

Quanto à originação e à qualificação de demandas, a proposta mais próxima da Camada de Tarefas da PASE é o LLM-Cure, de Assi, Hassan e Zou (2026). A abordagem usa modelos de linguagem para extrair as funcionalidades mencionadas em avaliações de usuários em lojas de aplicativos. Diante de uma reclamação sobre determinada funcionalidade, recupera avaliações bem pontuadas de aplicativos concorrentes sobre a mesma funcionalidade e sintetiza sugestões concretas de melhoria. A avaliação, feita sobre 1.056.739 avaliações de setenta aplicativos Android, indicou que, em média, 73% das sugestões geradas foram posteriormente implementadas pelos próprios desenvolvedores, o que evidencia a viabilidade de converter *feedback* informal e dados de concorrentes em propostas acionáveis.

Esse desenho corresponde, na PASE, à captação passiva do fluxo de agentes externos e à prospecção competitiva do fluxo da inteligência artificial. O LLM-Cure, entretanto, restringe-se a uma única fonte de insumos e encerra seu ciclo na recomendação ao desenvolvedor. Ele não contempla a normalização de entradas heterogêneas, a deduplicação contra o histórico de funcionalidades e de propostas descartadas, a verificação de coerência com as regras de negócio nem a atribuição de metadados de impacto, complexidade e confiança. A PASE estende essa linha ao tratar a mineração de *feedback* e a comparação com concorrentes como apenas duas das fontes de um funil unificado de filtragem progressiva, cujo produto é um *backlog* qualificado e pronto para priorização sob supervisão humana, e não uma lista de sugestões isoladas.

3.5 Personalização em Tempo de Execução: Interfaces Generativas e Maleáveis

A personalização em tempo de execução encontra seu precedente acadêmico mais direto em Cao, Jiang e Xia (2025), apresentado na CHI 2025. Os autores propõem interfaces generativas e maleáveis. Em vez de telas estáticas definidas em tempo de projeto, a interface é derivada de um modelo de dados orientado à tarefa, gerado por um LLM a partir do *prompt* do usuário e composto pelas entidades, relações e propriedades relevantes à atividade em curso. Esse modelo evolui à medida que o usuário refina sua intenção, e cada alteração se propaga para a interface e para os dados subjacentes. O protótipo Jelly e o estudo com usuários indicaram que os participantes valorizaram a combinação entre elementos estáticos previsíveis e elementos gerados sob demanda.

Esse resultado converge com a decisão da PASE de preservar o comportamento nativo e abrir à personalização apenas áreas delimitadas. A separação entre uma representação estruturada do estado, que o modelo gera, e sua renderização, que o sistema controla, é também o fundamento dos *payloads* declarativos das Zonas Dinâmicas. O trabalho de Cao, Jiang e Xia, porém, parte sempre de uma intenção explícita do usuário e corresponde apenas ao que a PASE denomina Zonas Dinâmicas Sob Demanda.

Não há, no trabalho de Cao, Jiang e Xia, equivalente às Zonas Dinâmicas Autônomas, acionadas pelo UPA a partir de telemetria, nem ao Avaliador de Zonas Dinâmicas, que decide, com base no padrão das demandas recebidas, quais módulos devem se tornar personalizáveis. Tampouco são tratados os mecanismos de governança próprios de um software comercial multiusuário, como contratos rígidos por zona com retorno ao padrão nativo, acesso a dados sob menor privilégio e filtragem prévia de injeção de *prompt*. A PASE incorpora a maleabilidade proposta pelos autores, mas a subordina a uma camada de governança que define onde, quando e sob quais limites a geração pode ocorrer.

3.6 Considerações Finais

Os trabalhos analisados mostram que cada frente da PASE possui precedentes, mas que nenhum deles a cobre por completo, como resume o Quadro 2. O HULA insere a pessoa na revisão de plano e de código, mas não na priorização nem na memória das decisões rejeitadas. O MetaGPT decompõe o ciclo de vida em papéis e artefatos, mas sem isolamento de execução, controle de custo ou auditoria da base de código. O LLM-Cure converte *feedback* disperso em sugestões, mas sem normalização de múltiplas fontes, deduplicação contra o histórico ou qualificação por metadados. E o trabalho de Cao, Jiang e Xia torna interfaces maleáveis pela intenção do usuário, mas sem gatilho por telemetria, sem critério para decidir quais áreas devem ser personalizáveis e sem contratos que limitem a geração.

Quadro 2 – Comparação entre os trabalhos relacionados e a Arquitetura PASE

Capacidade	**HULA**	**MetaGPT**	**LLM-Cure**	**Cao, Jiang e Xia**	**PASE**
Originação de demandas de múltiplas fontes	Não	Não	Parcial (uma fonte)	Não	Sim
Filtragem com memória de implementados e descartados	Não	Não	Não	Não	Sim
Decisão humana sobre o plano e sobre a entrega	Sim	Parcial (requisito inicial)	Não se aplica	Não se aplica	Sim
Dependências entre tarefas e controle de custo	Não	Não	Não	Não	Sim
Execução isolada e verificação por código	Parcial	Parcial	Não se aplica	Não	Sim
Personalização em tempo de execução governada por contrato	Não	Não	Não	Parcial (sem contrato)	Sim

Fonte: elaborado pelos autores (2026).

A contribuição da PASE, portanto, não se resume a integrar essas frentes. Dentro de cada camada, ela introduz mecanismos que os trabalhos relacionados não contemplam e, entre as camadas, subordina todos eles a um mesmo regime de autonomia supervisionada, rastreabilidade e contenção da generatividade.

4 Proposta: a Arquitetura PASE

4.1 Introdução

O Capítulo 2 mostrou que a transição para o software agêntico já produz ganhos concretos, mas ainda não dispõe de uma arquitetura que concilie a autonomia crescente dos sistemas de IA com a supervisão, o controle e o alinhamento humano. O Capítulo 3 mostrou que os trabalhos mais próximos tratam partes desse problema de forma isolada. Este capítulo apresenta a resposta deste trabalho a essa lacuna: a Arquitetura PASE (*Personalized Adaptive Software Ecosystem*).

A Arquitetura PASE é uma arquitetura de software híbrida humano-IA, voltada a estruturar o ciclo de vida completo de uma funcionalidade, da originação da ideia à sua integração e personalização em produção, sob um regime de autonomia supervisionada. Ela não persegue a automação total, nem reduz a IA a um papel apenas assistivo. Sua premissa condutora é uma divisão explícita de responsabilidades: os agentes de IA propõem, decompõem, executam e verificam, enquanto pessoas governam as decisões críticas e respondem por elas.

Cabe uma precisão conceitual. A PASE não é uma entidade inteligente que decide como se organiza. Ela é uma especificação arquitetural, definida pelos autores, que determina quais componentes existem, quais decisões cabem a modelos de linguagem, quais cabem a código determinístico e quais cabem a pessoas. As seções a seguir descrevem essa especificação no nível conceitual. Os detalhes de tecnologia, os parâmetros calibrados e as evidências de funcionamento são tratados no Capítulo 5.

4.2 Visão Geral e Princípios de Projeto

Quatro princípios de projeto sustentam a arquitetura e orientam todas as decisões descritas neste capítulo. Eles derivam diretamente dos requisitos consolidados na Seção 2.6 e funcionam como critério de desempate: quando duas soluções técnicas são possíveis, prevalece a que melhor preserva os quatro princípios. Cada um deles é apresentado a seguir.

O primeiro princípio é a **autonomia supervisionada** (*human-in-the-loop*). O julgamento humano é preservado nos pontos de maior incerteza e impacto, e não distribuído de forma difusa ao longo do fluxo. A arquitetura define de antemão onde a pessoa decide: na aprovação de uma tarefa candidata, na aprovação do plano técnico quando o risco o exige, na homologação da entrega e na publicação em produção. Essa escolha segue o

modelo de parceria humano-IA no qual a supervisão é indispensável para decisões sensíveis (BURTE, 2025).

O segundo princípio é a **contenção da generatividade**. A imprevisibilidade dos modelos de linguagem é aproveitada onde agrega valor, como na geração de ideias e na escrita de código, e bloqueada onde ameaça a integridade do sistema. A arquitetura combina camadas determinísticas de verificação com geração probabilística, na linha da integração neuro-simbólica apontada como direção promissora para a IA agêntica (ABOU ALI; DORNAIKA, 2025). Uma regra decorre desse princípio e atravessa todas as camadas: quando a decisão depende de um fato verificável, o modelo aponta a evidência e o código confere se ela existe.

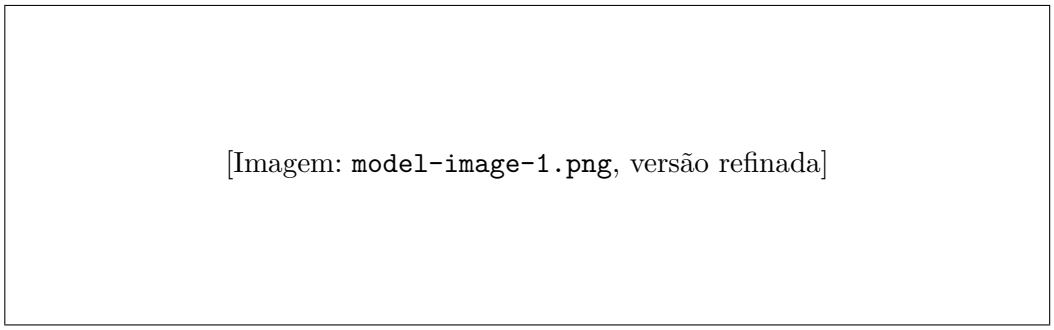
O terceiro princípio é a **rastreabilidade integral**. Cada decisão relevante, seja de um modelo, de uma regra ou de uma pessoa, gera um registro persistente que pode ser religado à demanda que a originou. A proveniência deixa de depender da memória da equipe e passa a ser uma propriedade do sistema. É esse registro que permite auditar, reverter e medir o comportamento da arquitetura ao longo do tempo.

O quarto princípio é o **alinhamento entre arquitetura e tarefa**. A eficácia de um sistema multiagente depende menos do número de agentes do que da adequação entre a estrutura de coordenação e a natureza do problema (KIM et al., 2026). Por isso, a PASE especifica contratos de entrada e saída entre as camadas, e não a estratégia interna de geração de cada uma. Uma camada pode usar um único modelo, vários agentes especializados ou uma cadeia de etapas, desde que respeite seu contrato.

Para operacionalizar esses princípios, a arquitetura organiza-se em três camadas sequenciais e complementares, apresentadas na Figura 2. A primeira é (i) a Camada de Tarefas (*Task Layer*), ponto de entrada que capta demandas de fontes heterogêneas, filtra redundâncias e incoerências e qualifica o que resta em tarefas candidatas. A segunda é (ii) a Camada de Orquestração (*Orchestration Layer*), plano de controle que prioriza, modela dependências, aplica os pontos de decisão humana e conduz cada tarefa até a entrega. A terceira é (iii) a Camada de Execução (*Execution Layer*), nível operacional que converte as decisões aprovadas em código verificado e, por meio do *User Personalization Agent* (UPA), entrega a personalização em tempo de execução sem expor o núcleo do sistema a código arbitrário.

As três camadas compartilham três bases de conhecimento, representadas na parte superior da Figura 2. O contexto geral do projeto reúne os documentos que definem o produto, como escopo, restrições, requisitos e regras de negócio. O histórico de tarefas registra o que já foi implementado ou está em andamento. O histórico de descartes guarda as propostas recusadas e os motivos de cada recusa. Essas bases não são estáticas: uma tarefa aprovada atualiza o histórico de tarefas e pode propor uma revisão do contexto, e uma tarefa recusada alimenta o histórico de descartes. Assim, a arquitetura aprende com

Figura 2 – Visão geral da Arquitetura PASE



Fonte: elaborado pelos autores (2026).

as próprias decisões sem que o sistema reescreva sozinho os critérios pelos quais é julgado, já que qualquer mudança no contexto exige aceite humano.

4.3 Camada de Tarefas (*Task Layer*)

A Camada de Tarefas transforma demandas brutas e dispersas em um *backlog* estruturado, qualificado e alinhado ao produto. Nela se materializa um equilíbrio central da arquitetura: abertura máxima na originação de demandas e rigor máximo na sua seleção. A camada admite insumos desestruturados ou de alta abstração e, em seguida, submete cada proposta a uma filtragem progressiva, na qual redundâncias, incoerências e itens de baixo valor são descartados antes de alcançar as camadas seguintes.

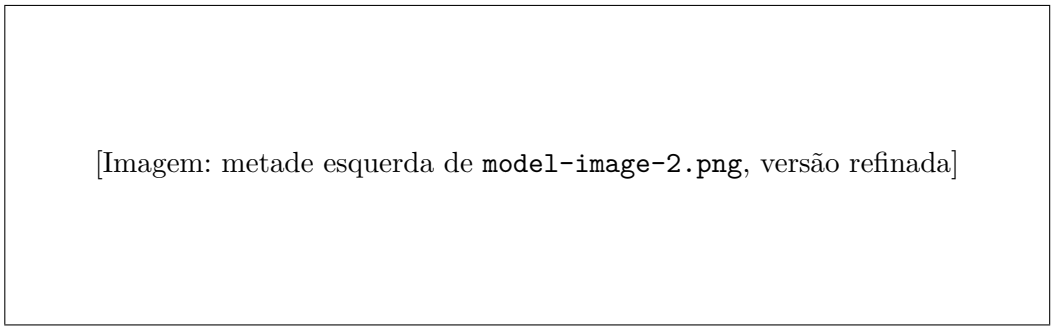
A camada organiza-se em quatro blocos encadeados, detalhados na Figura 3. Os dois primeiros preparam a demanda: (i) a Formação de Requisitos, que origina demandas a partir de usuários, equipe e IA, e (ii) o *Middleware* de Normalização, que converte cada insumo em um objeto padronizado. Os dois seguintes decidem seu destino: (iii) o Avaliador de Zonas Dinâmicas, que decide se a demanda pede uma funcionalidade nativa ou uma área personalizável, e (iv) o Motor de Filtragem de Tarefas, que valida a proposta contra a memória, o contexto e o valor esperado. O produto final é um conjunto de tarefas candidatas, anotadas por impacto, complexidade e confiança, pronto para a Camada de Orquestração.

4.3.1 Formação de Requisitos

A eficácia do ecossistema depende da diversidade dos insumos que alimentam a filtragem. Ao contrário do Motor de Filtragem, que prioriza rigor e consistência, a Formação de Requisitos é otimizada para alcance. Ela opera por três fluxos de originação: o fluxo de agentes externos, o fluxo de agentes internos e o fluxo da própria inteligência artificial.

O fluxo de agentes externos capta necessidades de quem usa o software. Na captação

Figura 3 – Camada de Tarefas da Arquitetura PASE



Fonte: elaborado pelos autores (2026).

ativa, uma interface simples, disponível em todas as telas, permite que o usuário sugira melhorias em linguagem natural e receba de volta a decisão tomada sobre a sugestão. A arquitetura prevê uma estratégia de engajamento em que contribuições convertidas em funcionalidades são recompensadas, com uma avaliação inicial feita por um modelo de baixa latência para filtrar ruído antes do processamento mais custoso.

Na captação passiva, a IA analisa tickets de suporte e outras interações não sensíveis do usuário para extrair intenções a partir de dores relatadas, convertendo reclamações informais em propostas estruturadas. A mineração de relatos de usuários e de registros de problemas como fonte de requisitos é apontada pela literatura como uma direção promissora, ainda pouco explorada (HOU et al., 2024; ZHANG et al., 2026). Como reclamações sobre a mesma dor costumam ser escritas de formas muito diferentes, a captação passiva agrupa por tema os tickets que relatam o mesmo problema antes de convertê-los, de modo que três pessoas relatando a mesma dor gerem uma única demanda, com o peso de três ocorrências.

O fluxo de agentes internos representa as inserções da equipe que conhece o software. As sugestões funcionais são demandas de novas funcionalidades, de natureza propositiva, e podem ser feitas por qualquer membro da equipe. Por isso, seguem o fluxo integral do Motor de Filtragem. As sugestões de engenharia, por sua vez, tratam de refatoração, débito técnico ou infraestrutura. Por serem decisões técnicas deliberadas, seguem da normalização diretamente para a qualificação, sem passar pelas camadas de memória e coerência. Esse desvio só é concedido a autores autorizados, pois a autorização já transfere o julgamento para uma pessoa qualificada.

O fluxo da inteligência artificial posiciona a IA como agente proativo na evolução do produto, capaz de antecipar necessidades sem depender de estímulos externos. Para manter a relevância das propostas, esse fluxo usa como guias o contexto geral do projeto, o mapa de funcionalidades já implementadas e o histórico de descartes. Ele se divide em três modos.

A geração exploratória usa o modelo configurado com alta variabilidade de amos-

tragem para propor funcionalidades que ultrapassam o escopo imediato, tratando como recurso criativo aquilo que em outros contextos seria um defeito. A análise de lacunas atua de forma analítica, cruzando módulos já implementados em busca de integrações ainda não exploradas. Por exemplo, a coexistência de um módulo de orçamento e um módulo de metas sugere um aviso que relacione o gasto do mês ao ritmo necessário para cumprir uma meta. A prospecção competitiva, por fim, compara periodicamente o produto com funcionalidades lançadas por concorrentes, com o objetivo de identificar defasagens, e não de replicá-las. Para reduzir vieses de um único modelo, a arquitetura prevê ainda a alternância periódica dos modelos usados nesse fluxo.

4.3.2 *Middleware* de Normalização

O *Middleware* de Normalização atua como tradutor universal entre fontes heterogêneas e o restante da camada. Um texto informal de cliente, um ticket de suporte ou uma sugestão técnica da equipe chegam em formatos e registros muito diferentes. A normalização extrai o núcleo semântico de cada proposta, remove ruídos de linguagem e a estrutura em um objeto padronizado, com título, descrição e módulo afetado. Etapas de preparação semelhantes, como a extração de informações-chave de relatos de defeitos, a estruturação de registros heterogêneos e a remoção de entradas duplicadas ou curtas demais, são recorrentes nos estudos que aplicam modelos de linguagem à engenharia de software (HOU et al., 2024; ZHANG et al., 2026).

Sem essa etapa, a comparação semântica das camadas seguintes seria comprometida pela variação na forma de escrita das fontes. Duas propostas idênticas em intenção, mas redigidas de modos distintos, poderiam escapar da detecção de duplicidade. A normalização garante que a memória da arquitetura compare ideias, e não estilos de escrita.

4.3.3 Avaliador de Zonas Dinâmicas

O Avaliador de Zonas Dinâmicas determina se uma demanda deve ser resolvida por uma funcionalidade nativa tradicional ou pela abertura de uma Zona Dinâmica, isto é, uma área do produto preparada para ser personalizada por usuário em tempo de execução. Para isso, o avaliador confronta a demanda com o histórico recente de pedidos sobre o mesmo módulo e decide entre duas linhas de solução.

Na primeira linha, a funcionalidade global estática, a dor identificada possui um padrão de solução homogêneo e aplicável à maior parte da base de usuários. Nesse caso, a arquitetura gera uma tarefa convencional para o desenvolvimento de código nativo permanente. Na segunda linha, a abertura de Zona Dinâmica, o módulo afetado concentra pedidos frequentes, mas com soluções muito divergentes entre usuários, típicos de uma cauda longa de necessidades individuais. Nesse caso, o avaliador bloqueia a criação de uma

funcionalidade estática e emite uma tarefa de parametrização, que transforma aquela área em um espaço personalizável, preservando seu comportamento padrão. A decisão de manter fixos os elementos estruturais da tela e abrir à adaptação apenas áreas delimitadas segue a recomendação de preservar âncoras visuais imutáveis durante mudanças de interface, para que o usuário não perca a orientação (ARORA et al., 2025).

A tarefa de parametrização delimita os limites de mutabilidade da área, define as regras de negócio e as restrições de segurança que valem ali e mantém o conteúdo agnóstico. Ela não descreve o que cada usuário verá, mas o que pode ou não ser personalizado. Quando a zona solicitada já existe, a demanda não gera nova tarefa de código: ela é encaminhada ao UPA, que ajusta a zona da pessoa que fez o pedido. Esse desvio evita que um pedido individual se torne uma mudança permanente para todos os usuários.

4.3.4 Motor de Filtragem de Tarefas

Todas as propostas que não são sugestões de engenharia autorizadas transitam pelo Motor de Filtragem de Tarefas. Ele atua como validador de segurança e viabilidade, e cada uma de suas camadas tem autonomia para descartar prematuramente propostas redundantes, incoerentes ou de baixo valor. O motor opera em três camadas sucessivas: (i) Memória Histórica, (ii) Coerência Sistêmica e (iii) Qualificação e Atribuição de Metadados.

A Camada de Memória Histórica realiza a análise de similaridade e a deduplicação. Ela compara a proposta com funcionalidades já implementadas, com tarefas em andamento e com propostas previamente descartadas, por meio de busca semântica em uma base vetorial. O objetivo é impedir a redundância e evitar que o produto reintroduza ideias que já se mostraram inadequadas. Uma duplicata não gera novo registro na base: ela incrementa a contagem de ocorrências do item existente. Essa escolha mantém a base densa e livre de ruído semântico, e ao mesmo tempo preserva a frequência com que um pedido aparece, sinal que o Avaliador de Zonas Dinâmicas utiliza.

A Camada de Coerência Sistêmica analisa se a proposta é compatível com o escopo e com as regras de negócio do produto, confrontando-a com os documentos do contexto geral do projeto. Ela tem autoridade de descarte imediato quando a incompatibilidade é irreversível, e o motivo do descarte cita o trecho do contexto violado. Quando a essência da proposta pode ser preservada com um recorte menor, a camada realiza um ajuste pontual. Nesse caso, a proposta ajustada é obrigatoriamente revalidada contra a Memória Histórica, porque o ajuste pode tê-la transformado em duplicata de algo existente. Se a compatibilização exigir alteração estrutural da ideia original, o motor opta pelo descarte, evitando um *backlog* preenchido por sugestões descaracterizadas.

A Camada de Qualificação e Atribuição de Metadados não descarta propostas. Ela as rotula com três variáveis que orientam a priorização. O impacto estimado expressa o

valor potencial para o usuário ou para o negócio, e a complexidade esperada estima a magnitude relativa do esforço (BURTE, 2025). O índice de confiança indica o grau de certeza do modelo sobre as duas estimativas anteriores. A camada atribui ainda uma faixa de risco à tarefa. Propostas com baixa confiança, ou com a combinação de baixo impacto e alta complexidade, recebem um marcador de incerteza, que sinaliza ao gestor humano a necessidade de análise mais criteriosa na Camada de Orquestração. O marcador aplica a tendência de concentrar a atenção humana no que mais importa, em vez de distribuí-la igualmente por todas as decisões (ANTHROPIC, 2026).

As propostas descartadas em qualquer camada são armazenadas na base vetorial de descartes com um resumo dos motivos determinantes. A arquitetura privilegia a densidade de informação: em vez de guardar o texto bruto, preserva o contexto crítico da rejeição. Por meio de geração aumentada por recuperação (*Retrieval-Augmented Generation*, RAG), técnica que ancora a resposta do modelo em informações recuperadas e verificadas para reduzir alucinações (ABOU ALI; DORNAIKA, 2025), apenas os descartes mais relevantes para a proposta em análise são levados ao modelo. Isso evita sobrecarga de contexto e transforma o histórico de descartes em uma base de conhecimento leve e consultável.

4.3.5 Módulo de Reavaliação Cíclica

Decisões de descarte não são permanentes, mas contextuais. Uma proposta recusada porque contrariava o escopo de ontem pode ser legítima diante do escopo de hoje. O Módulo de Reavaliação Cíclica é o processo em segundo plano que trata dessa deriva de contexto. Periodicamente, ou por acionamento humano, ele relê as propostas descartadas por incoerência e verifica se a restrição que justificou cada descarte ainda consta do contexto geral do projeto.

A verificação aplica o princípio da contenção da generatividade. Pedir a um modelo um veredito direto sobre se uma restrição ainda existe abre espaço para confabulação. Por isso, o modelo apenas aponta o trecho do contexto atual que sustentaria a manutenção do descarte, e a decisão cabe ao código, que confere se esse trecho existe literalmente no documento. Quando a restrição deixou de existir, a proposta é reintroduzida na filtragem com nova relevância. Assim, a arquitetura evita uma forma de estagnação por amnésia, em que boas ideias permaneceriam enterradas apenas porque foram rejeitadas em outro momento do produto.

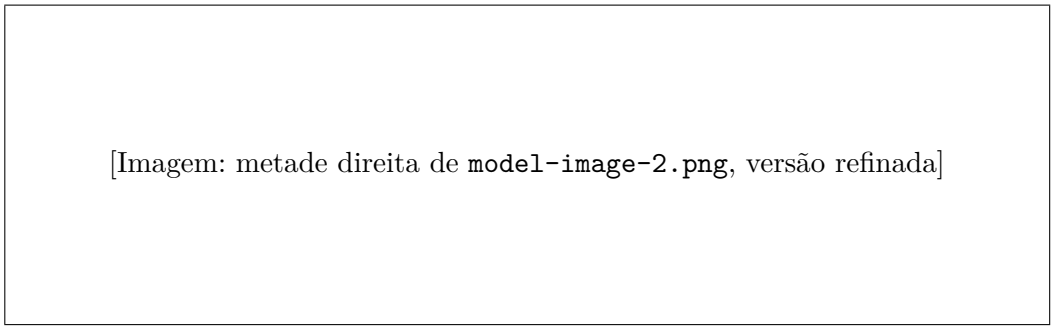
4.4 Camada de Orquestração (*Orchestration Layer*)

A Camada de Orquestração é o plano de controle da arquitetura e se posiciona entre a Camada de Tarefas e a Camada de Execução. Ela é ativada de forma reativa, pela chegada de novas tarefas candidatas ou pela ocorrência de incidentes em produção, e acompanha

cada demanda desde a priorização até a publicação. Nela se concentra a supervisão humana estrutural: o motor produz recomendações de priorização, sequenciamento e integração, mas a decisão sobre o que entra em execução, o que é aceito e o que chega à produção permanece com pessoas.

Para coordenar múltiplas tarefas sem conflito, a camada modela as interdependências entre elas em um grafo acíclico direcionado (*Directed Acyclic Graph*, DAG). A esse fluxo somam-se seis mecanismos de governança operacional, apresentados nesta seção. São eles (i) a priorização com decisão humana, (ii) o planejamento e o despacho guiados pelo grafo, (iii) o tratamento de falhas e a trava de custo, (iv) a homologação e a entrega em estágios, (v) o ciclo de vida por estados e (vi) o ciclo de auditoria recorrente. A Figura 4 detalha esses componentes e os da Camada de Execução.

Figura 4 – Camadas de Orquestração e de Execução da Arquitetura PASE



Fonte: elaborado pelos autores (2026).

4.4.1 Priorização e Decisão Humana

A priorização das tarefas candidatas usa os três metadados da qualificação, complementados por um fator de urgência. Esse fator é uma variável dinâmica calculada a partir de eventos externos, como incidentes ativos em produção, criticidade de segurança, prazos de acordos de nível de serviço ou metas de negócio. A recomendação favorece as tarefas que combinam maior impacto, maior urgência e menor complexidade.

A priorização é uma recomendação, e não uma ordem. O gestor humano pode alterar a sequência conforme a estratégia do projeto, usando o marcador de incerteza como alerta para as tarefas que pedem análise mais criteriosa. Manter essa decisão sob controle humano é coerente com a evidência de que a transformação organizacional com IA depende de redesenhar processos em torno do julgamento humano, e não de removê-lo (WORLD ECONOMIC FORUM; ACCENTURE, 2026). Aprovar uma tarefa candidata, recusá-la com justificativa ou editá-la são as três decisões disponíveis nesse ponto.

4.4.2 Planejamento, Dependências e Despacho

Após a aprovação de uma tarefa, a Orquestração solicita à Camada de Execução um Documento de Planejamento Técnico. Esse documento informa os arquivos que serão afetados, o roteiro de implementação, os critérios de aceite que a entrega deverá cumprir e as estimativas de custo e de tempo. Para tarefas de risco médio ou alto, ou quando o gestor assim escolhe, o plano passa por um ponto de decisão humana próprio antes de qualquer escrita de código. O plano é o momento mais barato para discordar: nele, a pessoa pode aprovar, devolver com orientações ou editar o roteiro, sem que nenhum recurso de execução tenha sido gasto.

Com o plano aprovado, a tarefa torna-se um novo nó do DAG de dependências, que mapeia a interdependência entre todas as tarefas aprovadas e ainda não concluídas. As dependências são representadas pelas arestas do grafo e podem ter três origens: a sobreposição de arquivos previstos nos planos, a proximidade semântica entre as tarefas e a indicação manual do gestor. Tarefas totalmente independentes são despachadas em paralelo. Quando há dependência, a decisão entre executar em paralelo ou aguardar os predecessores segue um limiar configurado pelo administrador, calculado a partir dos pesos acumulados das arestas. O grafo recusa, já na criação, qualquer aresta que formaria um ciclo.

Após a conclusão de uma tarefa, os planos das tarefas que dependem dela podem precisar de atualização. A arquitetura prevê que essa atualização seja automatizada a partir de um segundo limiar sobre os pesos de dependência. Para requisitos de alta complexidade, prevê também a decomposição do nó principal em subgrafos menores, de modo que falhas em subtarefas sejam isoladas e replanejadas sem interromper o fluxo geral (KIM et al., 2026).

4.4.3 Tratamento de Falhas e Trava de Custo

Se a Camada de Execução falhar, seja por erro de sintaxe, testes reprovados ou violação de alguma restrição, o motor aplica uma política automática de reexecução com número máximo de tentativas. Atingido esse limite sem sucesso, a tarefa é suspensa, junto com o ramo do grafo que depende dela, e o problema é reportado a uma pessoa. A partir da suspensão, o gestor pode retomar a tarefa, com ou sem orientações adicionais, replanejá-la do zero ou cancelá-la.

Acoplada à reexecução, a trava de custo (*budget guardrail*) protege o projeto contra ciclos de erro que consomem recursos sem convergir (SPARK, 2026). A verificação é preditiva: antes de cada nova chamada ao modelo, o motor compara o custo previsto dessa chamada, estimado a partir das chamadas anteriores da mesma tarefa, com a folga restante do orçamento. Uma verificação apenas retrospectiva, que só reagisse depois de o teto ser

ultrapassado, deixaria passar justamente a chamada que o estoura. O orçamento parte da estimativa do planejamento, acrescida de uma margem de tolerância, e pode ser ajustado pelo gestor.

4.4.4 Homologação e Entrega

Ao fim da execução, a Camada de Execução devolve à Orquestração um pacote com as alterações de código, os registros das verificações automáticas, o parecer da revisão independente e a telemetria de custo e tempo. A pessoa responsável revisa esse pacote e toma uma de três decisões. A primeira é (i) a recusa permanente, com justificativa, que envia a tarefa e o motivo ao histórico de descartes para que o erro não se repita. A segunda é (ii) a solicitação de ajuste, em que a pessoa aponta as modificações desejadas e a tarefa retorna à Execução para um novo ciclo. A terceira é (iii) a aprovação. Uma tarefa finalizada pode ainda ser refeita mais tarde, o que gera uma nova tarefa ligada à anterior e que carrega o plano, o parecer e o motivo da insatisfação.

Aprovar uma entrega não equivale a publicá-la. A aprovação integra a tarefa em um ramo de integração, onde ela é verificada junto com as demais tarefas aprovadas e pode ser testada funcionalmente. Somente tarefas aprovadas nesse ambiente entram em uma versão candidata, que passa por uma validação final completa. A publicação em produção é uma segunda decisão humana, registrada por escrito, que incorpora a versão ao ramo principal e a identifica com uma marca de versão. Essa separação distingue duas perguntas que costumam ser confundidas: “esta mudança está correta?” e “este conjunto de mudanças deve chegar aos usuários agora?”.

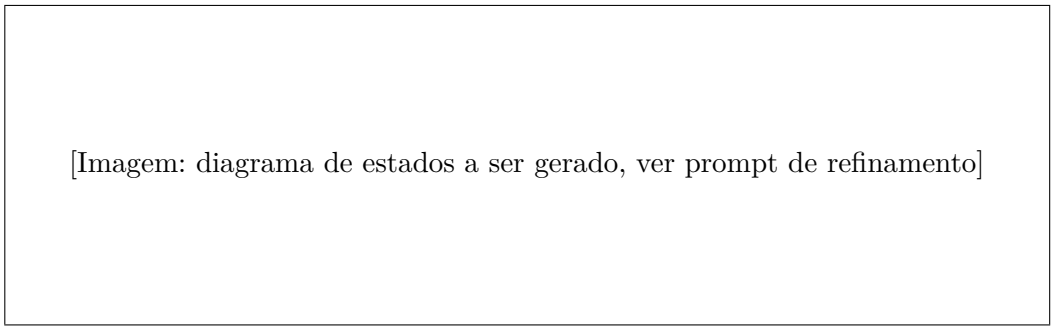
A aprovação também atualiza as bases de conhecimento. O item entregue passa a constar do histórico de tarefas, e a arquitetura propõe, a partir da entrega, uma atualização do contexto geral do projeto, que só é gravada se uma pessoa a aceitar. Uma reversão desfaz as três coisas juntas: o código, o registro de item implementado e a memória da proposta que o originou. Sem essa simetria, pedir novamente algo que foi revertido seria barrado como duplicata de algo que não existe mais.

4.4.5 Ciclo de Vida da Tarefa

O fluxo de controle descrito garante a consistência do ciclo de vida de cada tarefa, representado na Figura 5. Uma tarefa transita pelos estados Candidata, Em Planejamento, Aguardando Aprovação do Plano (quando exigida), Pronta para Despacho, Em Execução, Suspensa, Em Homologação e Finalizada. O estado Finalizada registra o desfecho, seja aprovação, recusa, cancelamento ou reversão. Depois da aprovação, a tarefa percorre ainda os estágios de entrega: integração, versão candidata e produção.

Toda transição gera um registro com origem, destino, momento e motivo. Os pontos

Figura 5 – Ciclo de vida de uma tarefa na Arquitetura PASE



Fonte: elaborado pelos autores (2026).

de decisão humana são sempre estados persistidos, e não processos travados à espera de resposta. Isso permite que uma tarefa aguarde uma decisão por minutos ou por dias sem consumir recursos, e que qualquer pessoa da equipe veja, a qualquer momento, quem ou o que a tarefa está esperando.

4.4.6 Ciclo de Auditoria e Saneamento Recorrente

Modelos de linguagem têm natureza probabilística. Ao realizarem sucessivas alterações granulares no repositório, podem provocar uma despadronização arquitetural gradual (KRAVCHUK-KIRILYUK; GRACIOLLI; AMIN, 2025) ou introduzir vulnerabilidades sutis, difíceis de perceber em revisões isoladas (HOU et al., 2024). Por isso, a Orquestração incorpora um Ciclo de Auditoria e Saneamento Recorrente, que opera em conjunto com a Camada de Execução para a manutenção proativa da base de código.

Periodicamente, ou por acionamento humano, o motor instrui a Execução a analisar o código do produto em busca de oportunidades de refatoração e de falhas de segurança. Cada achado gera uma nova sugestão de engenharia, marcada como originada da auditoria, que segue o mesmo fluxo das demais tarefas e passa pelos mesmos pontos de decisão humana. Desse modo, o ecossistema corrige os desvios produzidos por sua própria operação sem conceder a si mesmo um caminho privilegiado até a produção.

4.5 Camada de Execução (*Execution Layer*)

A Camada de Execução é o nível operacional da arquitetura, o plano em que as decisões aprovadas se convertem em software efetivamente integrado ao produto. Ela abrange dois planos complementares. O primeiro é o Executor de Tarefas, que recebe as diretrizes da Orquestração e as transforma em alterações de código verificadas. O segundo é o plano de personalização em tempo de execução, conduzido pelo UPA sobre as Zonas Dinâmicas.

Os dois planos compartilham o mesmo princípio de confinar a generatividade da IA a fronteiras explícitas e verificáveis, mas operam em regimes distintos. O Executor atua sobre o código-fonte permanente, sob controle da Orquestração e com latência de minutos. O UPA atua de forma autônoma sobre áreas previamente parametrizadas, em tempo de execução, com latência de segundos e sem alterar o repositório.

4.5.1 Executor de Tarefas: Fase de Planejamento Técnico

O Executor segue o princípio do isolamento. Ele não tem visibilidade sobre a priorização do *backlog* nem sobre as demais tarefas: recebe uma instrução de entrada, resolve o problema técnico e devolve resultados padronizados. A arquitetura fixa contratos de entrada e saída e restrições de ambiente, mas é agnóstica quanto à estratégia interna de geração (ANTHROPIC, 2026). O trabalho do Executor divide-se em duas fases separadas, a de planejamento técnico e a de execução, para que nenhuma linha de código seja escrita antes de haver um plano aprovado.

Na Fase de Planejamento Técnico, o Executor analisa o estado atual do repositório e produz o Documento de Planejamento Técnico. Esse documento contém quatro elementos. O primeiro é (i) o mapeamento de impacto, que identifica os arquivos e módulos a modificar, criar ou excluir, informação que a Orquestração usa para montar o grafo de dependências. O segundo é (ii) o roteiro de implementação, que descreve a estratégia técnica da alteração e serve de guia para a fase seguinte. O terceiro são (iii) os critérios de aceite, escritos do ponto de vista de quem usa o produto, que se tornam a régua de avaliação da entrega. O quarto são (iv) as estimativas de consumo de *tokens* e de tempo. Localizar corretamente onde uma mudança precisa ocorrer é, por si, uma das partes mais difíceis da resolução de problemas reais de software por modelos de linguagem (JIMENEZ et al., 2024), o que justifica tratar o mapeamento de impacto como artefato próprio e revisável.

Os critérios de aceite merecem destaque. Eles descrevem o efeito que o usuário deve observar, e não o mecanismo que o produz. Um critério como “ao clicar em usar tema escuro, as cores de fundo e de texto de todas as telas mudam” não é cumprido por uma entrega que apenas aplica uma classe a um elemento sem definir as cores correspondentes. Essa distinção é o que permite, mais adiante, avaliar se a entrega faz o que foi pedido, e não apenas se ela compila e passa nos testes.

4.5.2 Executor de Tarefas: Fase de Execução e Verificação

Após a aprovação do plano, o Executor inicia a escrita de código usando o roteiro de implementação como insumo principal, o que dispensa uma nova varredura completa do repositório e reduz tempo e consumo. Todo o processo ocorre em um ambiente isolado (*sandbox*), que clona o repositório e trabalha em um ramo exclusivo da tarefa (CHHABRA

et al., 2026). O ambiente não tem acesso de rede além do provedor do modelo de linguagem. A arquitetura não impõe uma lista rígida de arquivos permitidos: o Executor pode ajustar arquivos correlacionados caso surja uma necessidade imprevista, mas todo arquivo alterado fora do plano é registrado como desvio e destacado na homologação. Há, porém, fronteiras que não se negociam. Os componentes que implementam as garantias de segurança da própria arquitetura, como o código que valida as Zonas Dinâmicas, são inalcançáveis pelo Executor, e essa fronteira é verificada por código antes de qualquer escrita.

Dentro do ambiente isolado, o código gerado passa por um ciclo local de verificação e correção antes de interagir com qualquer parte externa. A verificação inclui análise estática, checagem de tipos, execução das suítes de teste e construção completa do produto para produção. Problemas detectados são corrigidos no próprio ambiente, sem onerar a Orquestração, até um limite de tentativas. Correções puramente mecânicas, como formatação e ordem de importações, são aplicadas por ferramentas determinísticas, e não pelo modelo.

A validação baseia-se no princípio da testabilidade autônoma, com três exigências verificadas por código (BURTE, 2025). A primeira é a preservação da rede de regressão: o inventário de testes existentes é obtido do repositório original, e não do ambiente de trabalho, e qualquer teste removido ou enfraquecido reprova a entrega. A segunda é que comportamento novo exige teste novo. A terceira é que a cobertura dos arquivos alterados não pode diminuir, o que impede que um teste trivial satisfaça formalmente a segunda exigência sem exercitar o código. Essas exigências são conferidas por código porque a simples instrução ao modelo não basta para garanti-las.

4.5.3 Revisão Independente e Retorno de Dados

Verificações mecânicas aprovadas não provam que a tarefa funciona. Uma entrega pode compilar, passar em todos os testes e, ainda assim, não fazer o que foi pedido, porque os testes conferem o mecanismo e não o efeito. Por isso, antes de chegar à pessoa, toda entrega passa por uma revisão independente, conduzida por um agente revisor distinto do que escreveu o código. O revisor avalia cada critério de aceite do ponto de vista do usuário e, para marcá-lo como atendido, deve citar o trecho do código que o cumpre e o trecho do teste que falharia se o critério não fosse cumprido.

As citações do revisor são conferidas por código: um trecho que não existe literalmente no arquivo anula o critério, ainda que o raciocínio esteja correto. O revisor também classifica cada alteração não solicitada pela tarefa. Alterações que mudam comportamento bloqueiam a entrega; alterações apenas cosméticas viram avisos para quem homologa, sem consumir novas rodadas de correção. Quando um critério depende de algo que o ambiente de teste não consegue verificar, como a cor efetivamente calculada pelo navegador, o revisor descreve o passo exato que a pessoa deve executar na tela para conferi-lo.

Encerradas as verificações, o Executor organiza os resultados em um pacote padronizado, enviado à Orquestração. O pacote contém o estado da operação, as alterações em formato de diferenças (*diff*), os registros das verificações e o parecer da revisão com as evidências conferidas. Contém também os arquivos alterados fora do plano, o histórico completo da interação com o modelo e a telemetria de consumo de *tokens* e de tempo. Com esse pacote, a pessoa que homologa vê não apenas o resultado, mas também como ele foi produzido, e pode assumir o controle da tarefa a qualquer momento. A arquitetura prevê ainda um ambiente temporário de visualização, no qual a entrega pode ser testada funcionalmente antes da decisão.

4.5.4 *User Personalization Agent* e Zonas Dinâmicas

Uma vez que uma tarefa de abertura de Zona Dinâmica é aprovada e implementada, a arquitetura delega a gestão dessa área a uma camada de tempo de execução controlada pelo *User Personalization Agent* (UPA). O UPA opera de forma autônoma e isolada do fluxo de requisitos globais. Ele observa a telemetria de interação de cada usuário e, ao identificar um padrão relevante em um módulo que possui Zona Dinâmica homologada, gera para aquele usuário uma configuração declarativa (*payload*) que personaliza a zona. O agente atua apenas sobre áreas já preparadas para isso e nunca sobre o restante da interface.

Os padrões que podem disparar essa avaliação incluem o uso intenso de uma área, cliques repetitivos que sugerem frustração, tempo excessivo para concluir tarefas e taxas elevadas de erro de validação. O uso de sinais de interação, como sequência de cliques, tempo gasto em tarefas e abandono de caminhos de navegação, é recorrente em sistemas adaptativos (TAN et al., 2025; BIENIEK; RAHOUTI; VERMA, 2024). A mesma literatura adverte, porém, que um sinal isolado é ambíguo e que a adaptação deve se apoiar em várias características do comportamento (TAN et al., 2025). Por isso, na PASE, a telemetria apenas aciona a avaliação, e a decisão de personalizar cabe ao agente, que pode concluir que não há nada a fazer.

Os sinais observados, a decisão do agente e o *payload* gerado são registrados, de modo que cada personalização pode ser auditada e revogada. Esse registro é o equivalente, no plano da personalização, da rastreabilidade que a Orquestração garante no plano do código.

O ponto central é o que o UPA não faz. Ele não gera código executável. O *payload* é um objeto de propriedades que preenche componentes do próprio produto, já existentes, revisados e testados, escolhidos de um catálogo fechado. Arora et al. (2025) descrevem uma arquitetura com dois caminhos: um rápido, que recupera componentes de interface pré-validados, e um lento, que gera código novo quando nenhum componente existente atende à necessidade. A PASE adota deliberadamente apenas o primeiro caminho em tempo

de execução e transfere o segundo para o fluxo governado da Camada de Execução, com planejamento, verificação e homologação. Dessa forma, o ecossistema alcança o que este trabalho denomina hiper-personalização, a adaptação individual e contínua da interface em tempo real, sem que a IA produza código arbitrário enquanto o produto está em uso. Para controlar o custo, cada combinação de usuário e zona tem um intervalo mínimo entre avaliações, de modo que o gasto com o modelo não cresça com a frequência de navegação.

As Zonas Dinâmicas admitem duas naturezas de acionamento. As Zonas Dinâmicas Autônomas são controladas de forma implícita pelo UPA, a partir da análise contínua de telemetria, e atuam apenas quando há uma necessidade identificada. As Zonas Dinâmicas Sob Demanda seguem uma abordagem baseada em intenção direta, na qual o usuário atua como coautor da interface ao expressar o que deseja em linguagem natural, de forma análoga à síntese dirigida descrita por Arora et al. (2025), em que novas funcionalidades da interface são geradas a partir de comandos explícitos do usuário. Na PASE, essa intenção explícita continua limitada ao catálogo e ao contrato da zona.

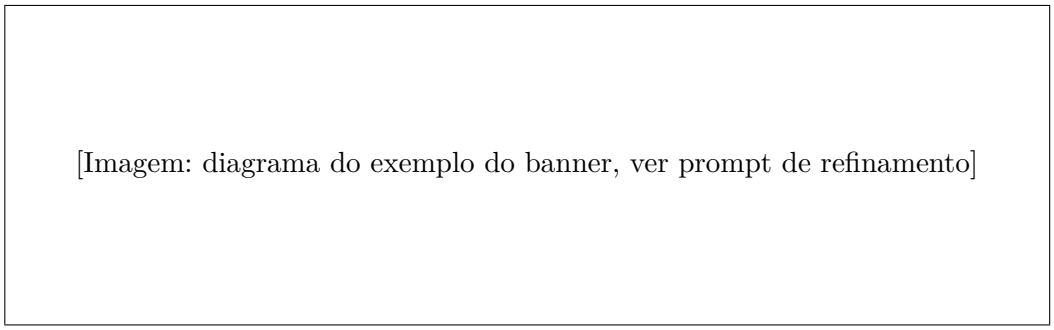
4.5.5 Governança das Zonas Dinâmicas

Para conter entradas inválidas, a arquitetura separa restrições abstratas (*Soft Constraints*) de contratos rígidos (*Hard Constraints*). O UPA atua no nível das restrições abstratas, usando o contexto do produto para decidir, de forma flexível, o que faz sentido mostrar a cada pessoa. A validação definitiva, contudo, é executada de forma determinística pelo código da própria zona, contra um contrato declarado. Se o *payload* violar o contrato, por usar um componente fora do catálogo, um parâmetro inválido ou um texto acima do limite, a zona aborta a renderização personalizada, exibe o componente nativo padrão e registra a violação.

A Figura 6 ilustra esse mecanismo com uma zona que exibe um aviso cujo contrato aceita apenas textos curtos e ligações para rotas internas do produto. Se o UPA, ou um comando do usuário, submeter um texto de quinhentos caracteres ou um endereço externo, a zona intercepta a inconformidade antes da renderização, descarta o conteúdo e mantém a visualização nativa. A validação ocorre em duas camadas, no servidor antes de persistir o *payload* e no cliente antes de exibi-lo, e a decisiva é a do cliente, porque é ela que decide o que chega à tela.

A abrangência das zonas pode ir além de componentes visuais isolados. Uma tarefa pode delimitar um processo de múltiplas etapas como ponto de extensão. Nesse caso, o *payload* não descreve apenas uma tela, mas uma máquina de estados com regras de navegação, validações de campos e condições de transição. Em vez de código de programação, como *JavaScript*, utiliza-se uma configuração estruturada, como JSON, o que preserva a flexibilidade do fluxo e protege a aplicação contra ataques de injeção de *scripts* (*Cross-Site Scripting*, XSS).

Figura 6 – Contrato rígido de uma Zona Dinâmica: payload válido e payload recusado



Fonte: elaborado pelos autores (2026).

As zonas operam ainda sob isolamento de escopo e menor privilégio. O acesso a dados é mediado por um kit de desenvolvimento estritamente tipado, que impede escrita arbitrária e acesso indevido à base de dados do produto. Nas zonas sob demanda, os *prompts* enviados pelo usuário passam por uma etapa de validação prévia, que intercepta intenções maliciosas e tentativas de injeção de *prompt* antes do processamento pelo agente (CHHABRA et al., 2026).

4.5.6 Interoperabilidade entre Zonas

Para viabilizar soluções mais complexas, a arquitetura prevê a comunicação entre diferentes Zonas Dinâmicas e o código estático por meio de um barramento local de eventos (*Event Bus*) no cliente da aplicação. Quando um componente dinâmico muda de estado, como um filtro personalizado, ele publica uma notificação no barramento. Outras zonas ou elementos nativos inscritos interceptam o evento e atualizam seu comportamento de forma sincronizada.

Na prática, essa estrutura permite que a interação do usuário com um filtro gerado em uma zona atualize uma tabela nativa e estática que escuta o barramento. A personalização deixa de ser um adorno isolado e passa a compor o comportamento da tela, sem que o contrato de nenhuma das partes seja violado.

4.6 Considerações Finais

Este capítulo apresentou a Arquitetura PASE em três camadas. A Camada de Tarefas disciplina a originação e a filtragem de demandas, convertendo insumos heterogêneos em um *backlog* qualificado e lembrando o que já foi feito e o que já foi recusado. A Camada de Orquestração concentra a supervisão humana em pontos definidos e coordena a execução por meio de um grafo de dependências, de travas de custo, de uma entrega em estágios e de um ciclo de auditoria. A Camada de Execução produz código em ambientes isolados,

verificado por máquina e revisado por um agente independente, e confina a personalização a zonas com contratos verificáveis.

Cada requisito da Seção 2.6 encontra resposta nesses mecanismos. O plano de controle da Orquestração, com custo observável e registro de cada transição, responde ao R1. A cobertura do ciclo desde a originação da demanda, os critérios de aceite e a revisão independente respondem ao R2. As Zonas Dinâmicas, com contrato rígido, selo de personalização e retorno ao padrão nativo, respondem ao R3. O isolamento da execução, a fronteira de escrita e os pontos de decisão humana sobre plano, entrega e publicação respondem ao R4. O pacote de homologação, com evidências conferidas e histórico completo da interação com o modelo, responde ao R5, ao dar a quem decide o contexto necessário para julgar.

O traço comum a todas as camadas é que a confiabilidade não é delegada ao comportamento do modelo. Onde uma garantia importa, ela é verificada por código ou decidida por uma pessoa. O Capítulo 5 descreve a implementação de uma prova de conceito dessa arquitetura e as evidências obtidas ao aplicá-la à evolução de um produto real.

5 Implementação e Validação

5.1 Introdução

Uma arquitetura proposta precisa demonstrar que funciona. Para verificar a viabilidade da Arquitetura PASE, foi construída uma prova de conceito (*Proof of Concept*, PoC) que implementa o núcleo das três camadas descritas no Capítulo 4 e o aplica à evolução de um produto real. O objetivo da validação é demonstrar que as camadas funcionam encadeadas, de ponta a ponta, sob governança humana e com rastreabilidade integral.

Este capítulo reúne implementação e validação porque as duas se informaram mutuamente. A implementação revelou falhas reais que motivaram ajustes na própria arquitetura, e esses ajustes são parte dos resultados. A Seção 5.2 descreve a prova de conceito. A Seção 5.3 apresenta o método de validação. As Seções 5.4 e 5.5 acompanham os dois fluxos centrais da arquitetura. A Seção 5.6 reúne os resultados quantitativos, e a Seção 5.7 os discute.

5.2 Implementação da Prova de Conceito

5.2.1 Produto-alvo e Arquitetura Técnica

A prova de conceito separa dois sistemas. O FinTrack é o produto-alvo, um painel financeiro pessoal composto por uma interface em *Next.js* e uma interface de programação em *FastAPI*, com módulos de painel inicial, transações, orçamentos, metas de economia, relatórios e suporte. O PASE é o motor que evolui o FinTrack: um núcleo em *FastAPI* sobre *PostgreSQL* com a extensão *pgvector*, organizado nas três camadas da arquitetura, acompanhado de um painel de controle para o operador humano. Os cinco serviços executam localmente em contêineres, e o FinTrack nunca é alvo de alterações fora do fluxo governado pelo PASE.

Três decisões de implementação materializam diretamente os princípios do Capítulo 4. A primeira é que toda chamada a modelo de linguagem passa por um cliente único, que registra modelo, *tokens*, duração e um identificador de rastreio, o que permite religar qualquer decisão do sistema à interação exata que a produziu. A segunda é que todo *prompt* é um arquivo versionado, e não um texto embutido no código; a PoC conta com quinze *prompts*, um para cada decisão delegada a modelo. A terceira é que cada decisão relevante gera um registro em banco, da proposta bruta à decisão humana final, incluindo cada transição de estado de cada tarefa.

A PoC usa modelos distintos por etapa, de acordo com a natureza do trabalho. As etapas do funil de filtragem, que classificam textos curtos, usam um modelo de menor custo (*gpt-4.1-mini*). A escrita de código usa um modelo especializado em programação (*gpt-5.3-codex*). A revisão independente usa um terceiro modelo (*gpt-5.5*), distinto do que escreveu o código. A busca semântica da Memória Histórica usa vetores gerados pelo modelo *text-embedding-3-small*. Essa separação é uma aplicação direta do princípio do alinhamento entre arquitetura e tarefa.

5.2.2 Correspondência entre a Arquitetura e a Implementação

A PoC não implementa a arquitetura por completo. A prioridade foi construir o núcleo que permite percorrer as três camadas de ponta a ponta, deixando para trabalhos futuros os componentes que ampliam a arquitetura sem alterar seu regime de governança. O Quadro 3 delimita o que foi construído e verificado em cada camada e o que permaneceu fora do escopo da prova de conceito.

Duas adaptações merecem registro. O isolamento da execução foi implementado por clone do repositório em diretório temporário e ramo exclusivo, e não por um contêiner efêmero por tarefa, o que preserva o efeito de isolamento do código, mas não o de isolamento de processo. Já a integração com o GitHub para abertura de *pull requests* reais foi implementada e configurada, mas a validação aqui relatada usou o modo local, no qual o próprio PASE desempenha o papel do *pull request*, com as mesmas exigências de validação e de aprovação humana por escrito.

5.2.3 Painel de Controle

O painel de controle concentra a supervisão humana e torna observável o funcionamento do motor. Nele estão as tarefas candidatas, ordenadas por prioridade, as tarefas em andamento, agrupadas por estado, o histórico das finalizadas e os descartes com seus motivos. As listas são atualizadas por eventos do próprio sistema, e não por consultas periódicas. As Figuras 7 a 10 apresentam quatro telas que evidenciam os princípios de rastreabilidade e de governança.

A tela de Contexto materializa as bases de conhecimento da arquitetura. O contexto do FinTrack é composto por dezenove documentos, entre eles escopo e restrições, requisitos, funcionalidades, arquitetura, banco de dados, interface, guia de engenharia, guia de ramos e registros de decisões de arquitetura. Cada edição gera uma nova versão, e cada etapa do ciclo lê apenas os documentos de que precisa. Quando uma tarefa é aprovada, o sistema propõe uma atualização do contexto, que só é gravada se uma pessoa a aceitar.

Quadro 3 – Componentes validados na PoC e escopo remanescente

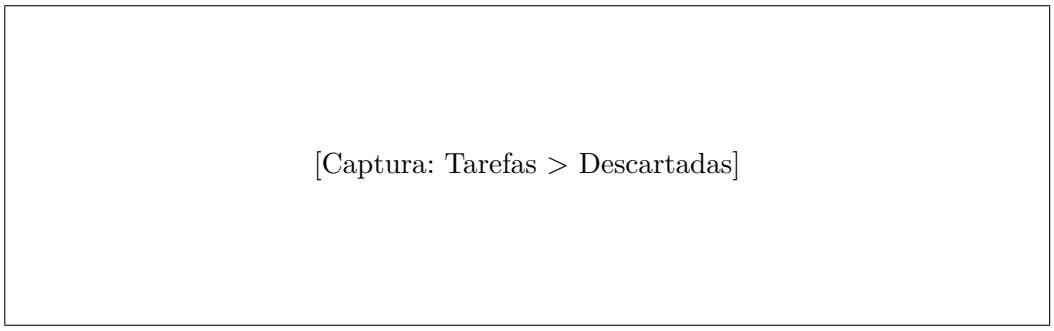
Camada	**Validado na PoC**	**Fora do escopo da PoC**
Tarefas	Seis origens de demanda (captação ativa e passiva, sugestões funcionais e de engenharia, geração exploratória e análise de lacunas); agrupamento de tickets por tema; normalização; Avaliador de Zonas Dinâmicas; Memória Histórica, Coerência Sistêmica e Qualificação; desvio autorizado para sugestões de engenharia; reavaliação de descartes	Prospecção competitiva; recompensa por contribuição; alternância periódica de modelos
Orquestração	Priorização com fator de urgência; ciclo de vida por estados com histórico de transições; pontos de decisão humana, inclusive sobre o plano; grafo de dependências com recusa de ciclos; reexecução e trava de custo preditiva; homologação; entrega em estágios (integração, versão candidata e produção); refazer e reverter tarefas; auditoria de código (por comando)	Pesos e limiares automáticos no grafo; subgrafos; replanejamento automático de dependentes; agendamento periódico dos ciclos de manutenção
Execução	Planejamento e execução separados; critérios de aceite; isolamento por clone e ramo exclusivo; verificação automática (análise estática, tipos, testes e construção para produção); preservação da rede de regressão, teste novo obrigatório e cobertura não decrescente; fronteira de escrita verificada por código; revisão independente com evidência conferida	Contêiner efêmero por tarefa; ambiente temporário de visualização
UPA	Seis Zonas Dinâmicas autônomas, uma por tela; *payload* declarativo; catálogo fechado de três componentes; validação em duas camadas; retorno ao componente nativo e registro de violação; encaminhamento de pedidos escritos à zona correspondente	Zonas conversacionais; barramento de eventos entre zonas; fluxos declarativos de múltiplas etapas; filtro dedicado de injeção de *prompt*

Fonte: elaborado pelos autores (2026).

5.3 Material e Métodos

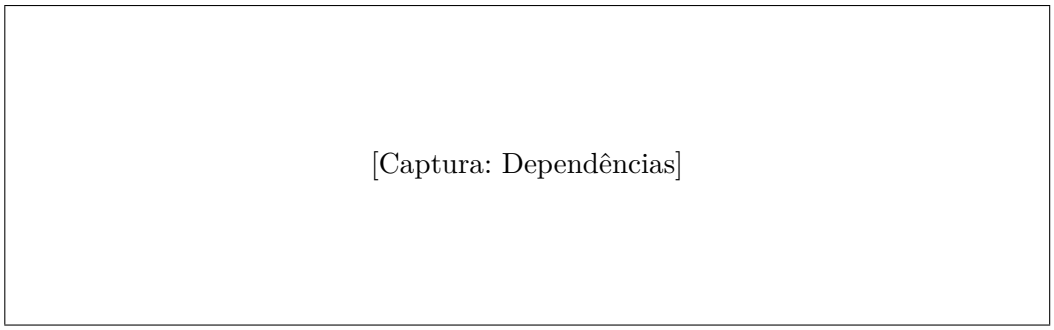
A validação adota a estratégia de estudo de caso: a arquitetura é aplicada à evolução de um produto e seu comportamento é observado em execuções reais, com modelos de linguagem reais, e não simulados. O período de observação vai de 16 de setembro a 10 de

Figura 7 – Descartadas: propostas barradas pelo funil por duplicidade ou incoerência, com o motivo de cada descarte



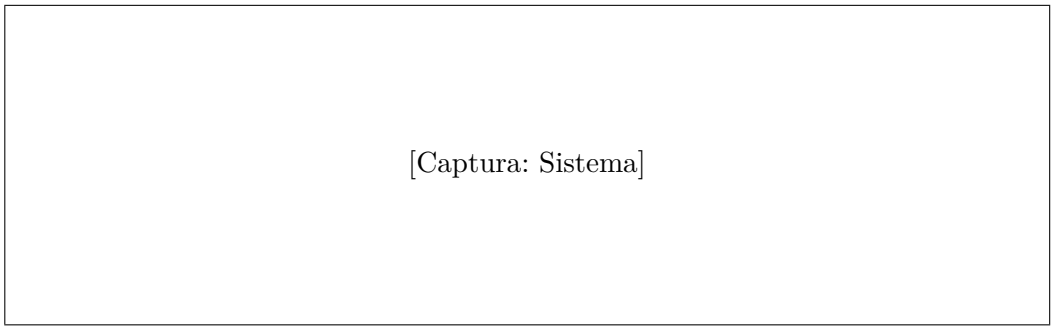
Fonte: elaborado pelos autores (2026).

Figura 8 – Dependências: grafo das tarefas, com nós coloridos por estado e arestas de dependência



Fonte: elaborado pelos autores (2026).

Figura 9 – Sistema: fila de execução, memória vetorial, consumo acumulado, travas e *prompts* versionados

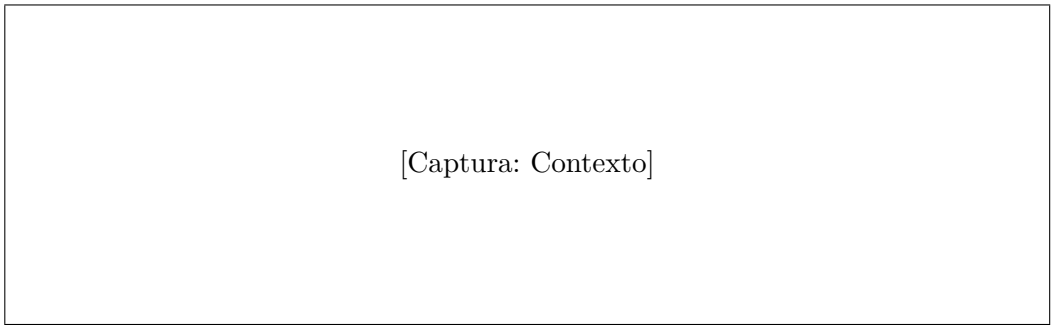


Fonte: elaborado pelos autores (2026).

outubro de 2026, do primeiro *commit* da PoC à última execução registrada. A validação apoia-se em quatro fontes de evidência complementares.

A primeira fonte são os registros das execuções reais, mantidos no banco do PASE e no histórico do repositório. Eles incluem cada proposta recebida, cada descarte com seu motivo, cada tarefa e suas transições de estado, cada execução com seu consumo e sua duração e cada evento do fluxo de entrega. A segunda fonte são oito roteiros de

Figura 10 – Contexto: documentos do produto versionados no banco e as etapas do ciclo que leem cada um



Fonte: elaborado pelos autores (2026).

demonstração executáveis, cada um projetado para provar uma propriedade específica da arquitetura, apresentados no Quadro 4. A terceira são as suítes de testes automatizados, que somam 533 testes: 433 no núcleo do PASE, 53 na interface de programação do FinTrack e 47 na sua interface de usuário. A quarta são três métricas definidas antes da construção da PoC, para avaliar o equilíbrio entre autonomia e supervisão.

Quadro 4 – Roteiros de demonstração e as propriedades que verificam

Roteiro	**Propriedade verificada**	**Princípio**
D1	Uma demanda em linguagem natural vira código integrado, passando por decisão humana	Autonomia supervisionada
D2	Duplicatas e propostas incoerentes são barradas, com motivo e contagem	Rastreabilidade integral
D3	A IA gera propostas próprias, nos modos exploratório e de análise de lacunas	Autonomia supervisionada
D4	A personalização autônoma aparece para uma persona e não para a outra	Contenção da generatividade
D5	*Payloads* maliciosos não chegam à tela: retorno ao nativo e violação registrada	Contenção da generatividade
D6	O teto de tentativas e o teto de custo interrompem a execução de fato	Contenção da generatividade
D7	Tickets repetidos sobre a mesma dor viram uma demanda de Zona Dinâmica	Alinhamento entre arquitetura e tarefa
D8	A auditoria propõe refatoração e a reavaliação reabre um descarte obsoleto	Rastreabilidade integral

Fonte: elaborado pelos autores (2026).

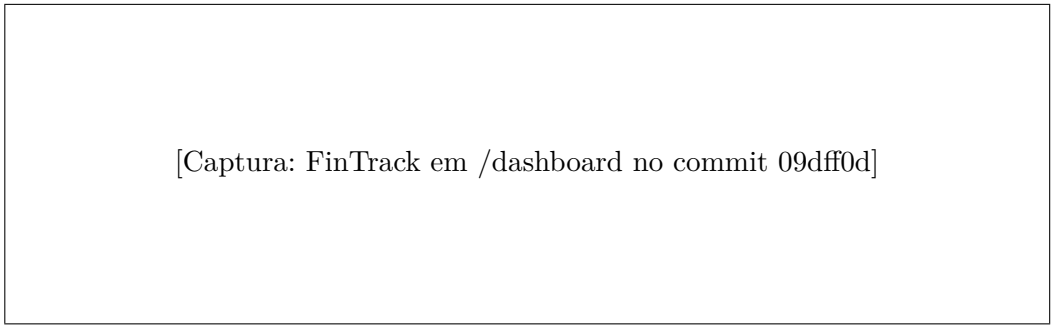
As três métricas foram definidas da seguinte forma. A taxa de intervenção humana

é a proporção de decisões de ajuste ou de recusa sobre o total de entregas que chegaram à homologação. A densidade de retrabalho é o número médio de execuções por tarefa finalizada. A incidência de regressões é o número de tarefas cujos testes falharam em uma execução depois de terem passado em uma execução anterior da mesma tarefa. As três são calculadas a partir de arquivos exportados pelo próprio sistema, e não de anotações manuais.

5.4 Fluxo 1: da Demanda ao Código em Produção

O primeiro fluxo acompanha uma funcionalidade real, o modo escuro do FinTrack, registrada como tarefa nº 6 do PASE, da demanda em linguagem natural até a publicação na versão 1.0.0 do produto. A escolha não é casual. O modo escuro foi a funcionalidade que mais exigiu da arquitetura, e as tentativas anteriores, discutidas na Seção 5.7, motivaram parte dos mecanismos descritos no Capítulo 4. A Figura 11 mostra o produto antes da solicitação.

Figura 11 – FinTrack antes da solicitação: interface apenas em tema claro, sem controle de tema

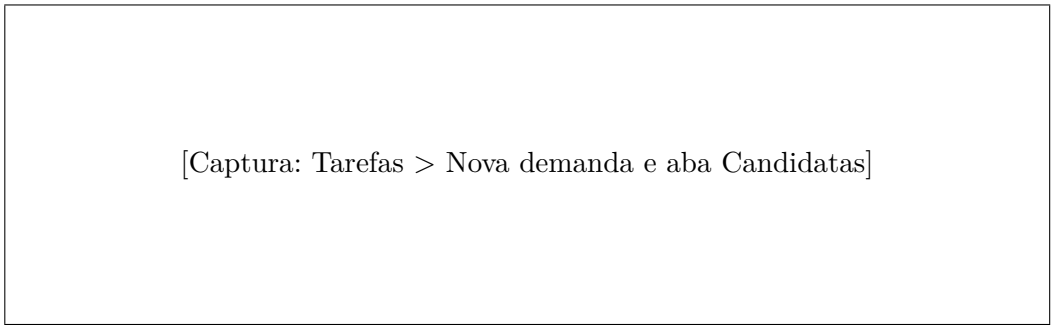


Fonte: elaborado pelos autores (2026).

A demanda entra no painel em texto livre e atravessa o Motor de Filtragem: normalização, Memória Histórica, com busca vetorial contra itens implementados, em andamento e descartados, Coerência Sistêmica, frente aos documentos de contexto, e Qualificação, por impacto, complexidade, confiança e risco. O resultado é uma tarefa candidata priorizada, que aguarda a decisão do gestor (Figura 12).

Aprovada a candidata, a cadeia segue de forma autônoma até o próximo ponto de decisão humana. Na Fase de Planejamento, o Executor mapeou sete arquivos a alterar e definiu cinco critérios de aceite antes de escrever qualquer código. Como a tarefa refazia uma tentativa anterior, o plano passou pelo ponto de decisão humana próprio e só foi liberado depois de aprovado (Figura 13). Na Fase de Execução, o Executor trabalhou em clone isolado e ramo exclusivo, e a entrega só chegou à pessoa depois de passar pela verificação automática: análise estática, tipos, testes, construção para produção,

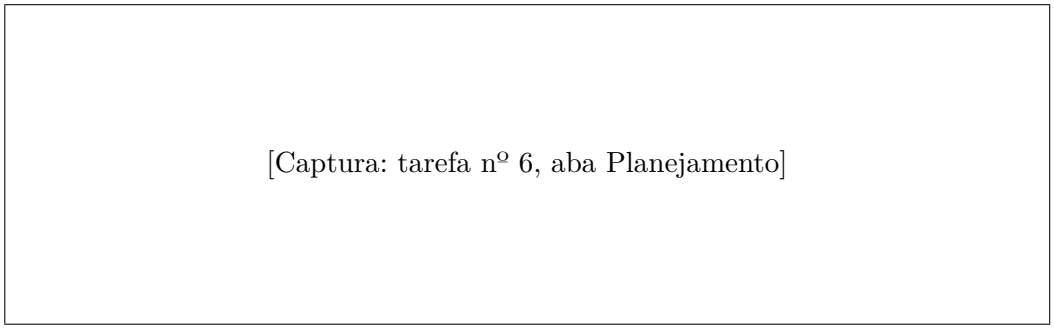
Figura 12 – Painel do PASE recebendo a solicitação: demanda em linguagem natural e tarefa candidata qualificada, aguardando aprovação ou recusa



Fonte: elaborado pelos autores (2026).

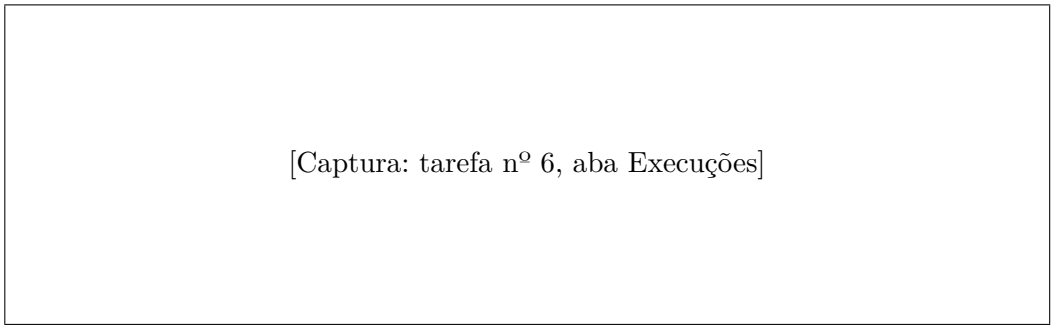
preservação dos testes existentes, teste novo para comportamento novo e cobertura não decrescente (Figura 14).

Figura 13 – Documento de Planejamento Técnico: arquivos previstos, critérios de aceite (C1 a C5) e trilho do ciclo de vida da tarefa



Fonte: elaborado pelos autores (2026).

Figura 14 – Execução registrada: diferenças de código, registro das verificações automáticas e transcrição integral da interação com o modelo

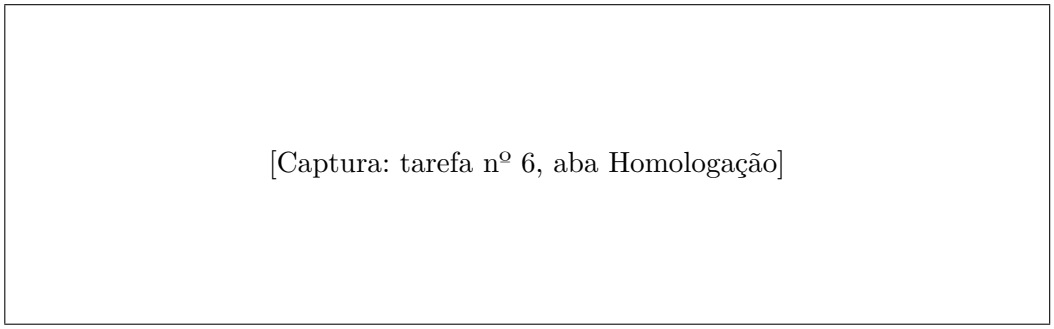


Fonte: elaborado pelos autores (2026).

A entrega definiu a paleta escura, adicionou o botão de alternância e persistiu a preferência por persona, com um teste para cada um dos cinco critérios de aceite. Um agente revisor separado conferiu cada critério citando o código e o teste correspondentes, e

o sistema verificou por código que as citações existem. Só então o gestor decidiu (Figura 15). A execução consumiu 113.131 *tokens* e durou cerca de seis minutos, da liberação do plano à chegada na homologação.

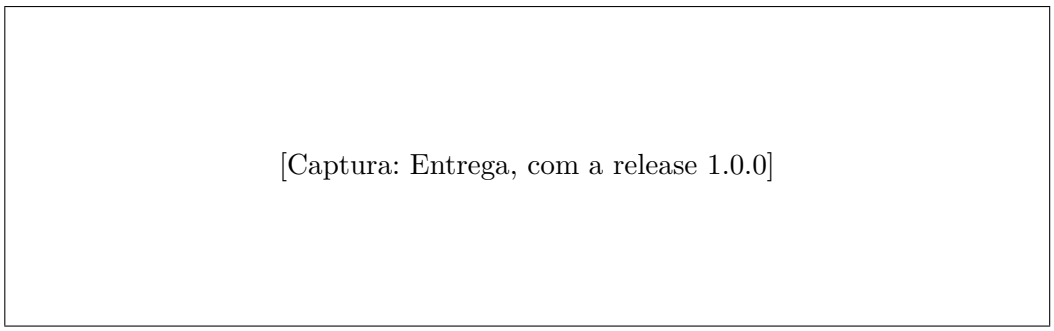
Figura 15 – Homologação: parecer da revisão independente por critério de aceite e decisão do gestor (aprovar, ajustar ou recusar)



Fonte: elaborado pelos autores (2026).

Aprovar não equivaleu a publicar. A tarefa foi integrada ao ramo de integração, onde a verificação contínua passou. Em seguida, foi aprovada funcionalmente, com uma nota registrada pela pessoa responsável, incluída na versão candidata 1.0.0 e validada com as suítes completas. Somente então foi publicada no ramo principal, mediante nova aprovação humana por escrito, e marcada com a versão 1.0.0 (Figura 16). O histórico do repositório preserva essa cadeia com autoria do agente, nos *commits* 7c810b6, 0d6050a e 0ff8483, e a funcionalidade passou a valer no produto em execução (Figura 17).

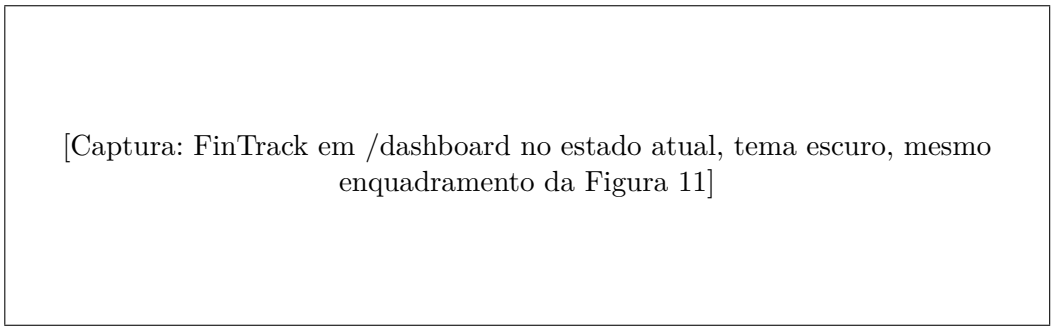
Figura 16 – Tela Entrega: tarefa no ramo de integração, versão 1.0.0 validada e publicação no ramo principal sob aprovação humana



Fonte: elaborado pelos autores (2026).

O Quadro 5 resume a sequência de eventos registrada pelo sistema para essa tarefa. Os intervalos entre os eventos que dependem de pessoas são maiores que os intervalos entre os eventos automáticos, o que é esperado em um regime de autonomia supervisionada: a máquina não espera, e a pessoa decide no seu próprio tempo.

Figura 17 – FinTrack após a entrega: botão “Usar tema escuro” e paleta escura aplicada a todas as telas



Fonte: elaborado pelos autores (2026).

Quadro 5 – Linha do tempo da tarefa nº 6, conforme os registros do sistema (8 de outubro de 2026)

Horário	**Evento**	**Responsável**
17h34	Tarefa criada como refação da tarefa nº 4, com o motivo da insatisfação anterior	Pessoa
17h37	Plano pronto: sete arquivos e cinco critérios de aceite, aguardando aprovação	Agente
18h21	Plano aprovado e execução iniciada	Pessoa
18h27	Entrega verificada e revisada, pronta para homologação	Agente
18h37	Entrega aprovada e integrada no ramo de integração	Pessoa
18h38	Verificação contínua do ramo de integração aprovada	Sistema
19h06	Aprovação funcional registrada	Pessoa
19h09	Versão candidata 1.0.0 criada a partir do ramo principal e tarefa incluída	Pessoa
19h10	Validação final da versão candidata aprovada	Sistema
19h13	Versão 1.0.0 publicada no ramo principal, com marca de versão	Pessoa

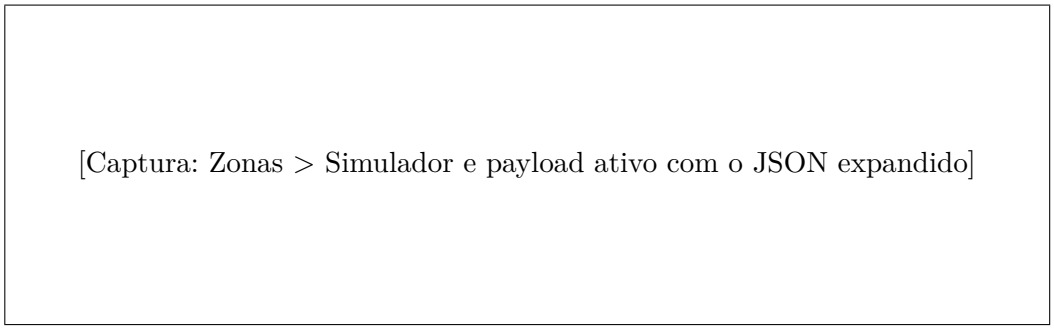
Fonte: elaborado pelos autores (2026), a partir dos registros de transição de estado e de entrega do PASE.

5.5 Fluxo 2: Personalização em Tempo de Execução

O segundo fluxo valida o UPA. Sem tocar o repositório e sem decisão humana, ele personaliza as Zonas Dinâmicas do FinTrack, que são seis, uma por tela, preenchendo propriedades de um catálogo fechado de três componentes: uma lista de atalhos, um cartão de resumo e uma faixa de aviso. A telemetria de navegação aciona a regra da zona, o UPA gera um *payload* declarativo e o servidor o valida contra o contrato antes de persisti-lo

(Figura 18). O contrato é um único documento canônico, espelhado no cliente; um teste automatizado compara os dois, campo a campo, para impedir que divirjam.

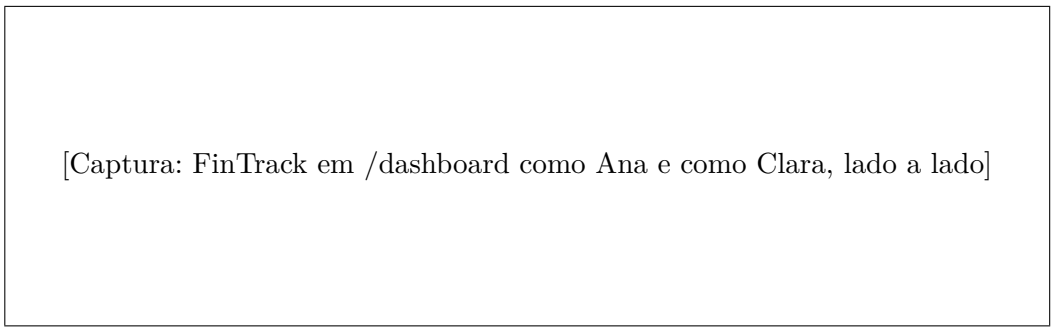
Figura 18 – Simulador do painel: o uso intenso de relatórios pela persona Ana aciona a zona do painel inicial e o UPA gera um payload declarativo



Fonte: elaborado pelos autores (2026).

No cliente, uma segunda validação, a decisiva, confronta o *payload* com o contrato rígido antes de exibi-lo. A personalização aparece apenas para quem a motivou (Figura 19). Cada zona observa uma rota e um número mínimo de visitas em uma janela de sete dias, e cada combinação de pessoa e zona respeita um intervalo de vinte e quatro horas entre avaliações, de modo que o custo da personalização não cresça com a frequência de navegação.

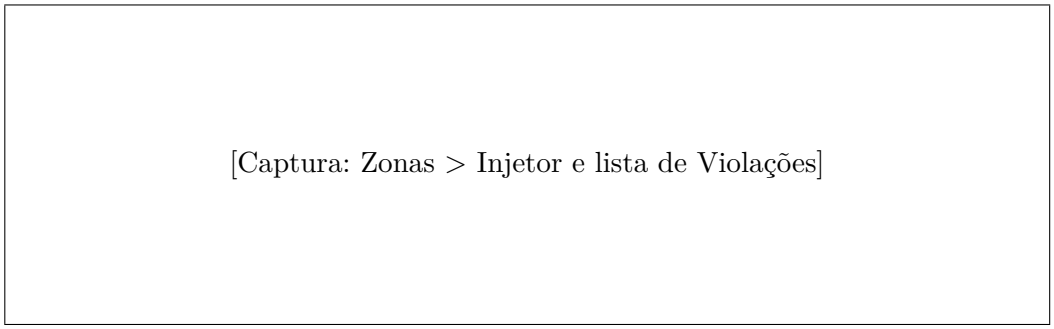
Figura 19 – Personalização em execução: Ana vê o componente gerado pelo UPA, com o selo “personalizado”, enquanto Clara mantém o componente nativo



Fonte: elaborado pelos autores (2026).

A defesa foi testada no pior caso: *payloads* maliciosos gravados diretamente no banco, contornando a validação do servidor. Os três casos foram um rótulo acima do limite de quarenta caracteres, um *link* para endereço externo e um componente fora do catálogo, nomeado `iframe_injector`. O cliente recusou os três, manteve o componente nativo e registrou cada violação com o motivo e o conteúdo bruto (Figura 20). Em nenhum momento a IA gera código executável.

Figura 20 – Defesa em profundidade: payload com link externo injetado sem validação do servidor, recusado pelo cliente e registrado como violação



Fonte: elaborado pelos autores (2026).

5.6 Resultados Obtidos

No período de observação, a PoC registrou seis tarefas, dez execuções do Executor e oitenta e cinco chamadas a modelos de linguagem, que somaram cerca de 1,19 milhão de *tokens*. O histórico do repositório registra vinte e três *commits* com autoria do agente do PASE, entre implementações, integrações, reversões e a publicação da versão 1.0.0. A Tabela 1 apresenta o desfecho de cada tarefa.

Tabela 1 – Tarefas executadas pela PoC e seus desfechos

Tarefa	**Demanda**	**Execuções**	***Tokens***	**Desfecho**
1	Editar a data de uma transação diretamente na lista	1	53.457	Aprovada e mantida
2	Implementar modo escuro no sistema	1	76.154	Aprovada e revertida depois
3	Implementar modo escuro no sistema	4	114.408	Interrompida pela trava de custo e cancelada
4	Implementar modo escuro no FinTrack	1	109.720	Aprovada e revertida depois
5	Exportar transações em formato CSV	2	485.851	Reprovada pela revisão independente e cancelada
6	Implementar modo escuro no FinTrack (refação da nº 4)	1	113.131	Aprovada, publicada na versão 1.0.0

Fonte: elaborado pelos autores (2026), a partir dos registros de execução do PASE.

No funil de filtragem, as nove propostas recebidas no período tiveram desfechos explícitos. Quatro foram descartadas por incoerência com o contexto do produto, e cada motivo cita a linha do documento de escopo que a proposta violava: mineração de

criptomoedas, editor de vídeo, rede social entre usuários e metas de gasto por categoria. As outras cinco tornaram-se tarefas. Em um ciclo anterior de demonstração, registrado em 19 de setembro de 2026, o funil havia produzido treze descartes, oito por incoerência e cinco por duplicidade, e o cliente havia recusado os três *payloads* maliciosos descritos na Seção 5.5.

A Tabela 2 apresenta as três métricas de validação definidas na Seção 5.3, acrescidas de uma quarta, a taxa de aprovações revertidas, que se mostrou necessária para interpretar a primeira. As métricas exigem cuidado de leitura e são discutidas na seção seguinte.

Tabela 2 – Métricas de validação calculadas sobre as tarefas do período

Métrica	**Definição**	**Valor**
Taxa de intervenção humana	Ajustes ou recusas sobre entregas que chegaram à homologação	0 de 4 (0%)
Aprovações revertidas	Entregas aprovadas na homologação e revertidas depois, pela pessoa	2 de 4 (50%)
Densidade de retrabalho	Execuções por tarefa finalizada	10 / 6 = 1,67
Incidência de regressões	Tarefas com testes que falharam depois de terem passado na mesma tarefa	0

Fonte: elaborado pelos autores (2026).

Dois resultados adicionais quantificam fenômenos relevantes para o desenho da arquitetura. O primeiro é a imprecisão da estimativa de custo feita pelo modelo no planejamento: para a tarefa nº 6, o plano estimou 7.800 *tokens*, e a execução consumiu 113.131, cerca de quatorze vezes mais. O segundo é o consumo das execuções que não convergiram: a tarefa nº 5 consumiu 485.851 *tokens* em duas execuções sem chegar a uma entrega aprovável, o maior consumo do período.

5.7 Discussão

Os dois fluxos demonstram que o núcleo da arquitetura funciona. Uma demanda em linguagem natural percorreu as três camadas e chegou à produção com decisão humana em cada ponto crítico, e a personalização em tempo de execução operou sem expor o núcleo do produto a código arbitrário. Mais do que demonstrar o caminho de sucesso, porém, a validação expôs falhas reais, e a forma como a arquitetura foi ajustada em resposta a elas é o resultado mais informativo deste trabalho.

O caso do modo escuro ilustra o princípio da contenção da generatividade na prática. A funcionalidade exigiu quatro tentativas. A tarefa nº 2 passou na checagem de tipos e nos testes e foi aprovada, mas quebrou o aplicativo inteiro, porque introduziu estado em um componente que o compilador do *framework* exige que seja estático. Nem a checagem de

tipos nem os testes cobriam essa regra, e a construção completa para produção passou a integrar a verificação. A tarefa nº 4 passou em todas as verificações e também foi aprovada, mas não fazia nada visível: aplicava uma classe ao documento, testava que a classe estava lá e nunca definia uma cor escura. Os testes provavam o mecanismo, e ninguém conferia o efeito.

Esse segundo episódio explica a métrica mais reveladora da Tabela 2. A taxa de intervenção humana na homologação foi nula, e ainda assim metade das entregas aprovadas foi revertida depois, quando a pessoa observou o produto em execução. A leitura correta não é que o gate humano seja desnecessário, mas que um gate humano diante de um *diff* plausível e de verificações verdes é insuficiente. A resposta da arquitetura foi tornar explícito o que deve ser conferido, com os critérios de aceite definidos no planejamento, e quem confere antes da pessoa, com a revisão independente e a evidência verificada por código. Depois dessa mudança, a tarefa nº 5 foi barrada pela revisão antes de chegar à homologação, e a tarefa nº 6 foi aprovada e permaneceu em produção. A amostra é pequena, e o resultado não autoriza generalização, mas indica a direção correta do mecanismo.

A tarefa nº 3 ilustra a trava de custo. Duas retomadas solicitadas pelo gestor foram interrompidas em segundos, sem consumir nenhum *token*, porque o custo previsto da próxima chamada ultrapassava a folga do orçamento. A tarefa nº 5 mostra o outro lado do mesmo problema: sem um limite apropriado, ciclos de correção que não convergem consomem recursos rapidamente. Somada à subestimação de cerca de quatorze vezes na tarefa nº 6, essa evidência confirma que a estimativa do modelo serve como artefato de planejamento, mas não pode ser usada sozinha como teto de gasto.

Todas essas lições têm a mesma forma, que o Quadro 6 resume. Em cada caso, uma garantia que inicialmente dependia de instrução ao modelo passou a ser verificada por código ou decidida por uma pessoa. Esse padrão é coerente com a literatura que aponta a combinação entre camadas determinísticas e geração probabilística como caminho para sistemas agênticos confiáveis (ABOU ALI; DORNAIKA, 2025), e com a evidência de que sistemas multiagente acumulam erros quando não há verificação centralizada (KIM et al., 2026).

A validação também tem limites claros. O número de tarefas é pequeno, o produto-alvo é de porte reduzido e os operadores humanos foram os próprios autores, o que impede conclusões estatísticas sobre produtividade ou sobre a carga cognitiva da supervisão. Esses limites, e os que decorrem do escopo da revisão de literatura, são discutidos na Seção 6.2.

5.8 Considerações Finais

Este capítulo descreveu a implementação de uma prova de conceito da Arquitetura PASE e a validação de seus dois fluxos centrais. O fluxo de evolução de código levou

Quadro 6 – Ajustes motivados pela validação: do previsto ao construído

Previsto inicialmente	**Ajuste realizado**	**Falha que o motivou**
Confiar que o modelo preserva os testes existentes	Inventário de testes obtido do repositório original e conferido por código	Na primeira execução real, o Executor sobrescreveu um teste existente, e a verificação passou porque rodou a suíte adulterada
Verificações mecânicas bastam antes da pessoa	Critérios de aceite e revisão independente por outro modelo	O modo escuro passou em todas as verificações sem definir nenhuma cor
O revisor decide se a entrega cumpre o pedido	O revisor cita o trecho; o código confere se ele existe	Vereditos diretos de modelo sobre fatos verificáveis tendem a confabulação
Toda alteração colateral reprova a revisão	Só o que muda comportamento bloqueia; o cosmético vira aviso	Alterações de aspas e de comentários consumiram quatro rodadas de correção em uma tarefa que cumpria os cinco critérios
Checagem de tipos e testes bastam no frontend	Construção completa para produção na verificação	Uma entrega passou nas duas e quebrou o aplicativo inteiro
Orçamento conferido contra o gasto acumulado	Orçamento conferido contra o custo previsto da próxima chamada	A verificação retrospectiva deixava passar justamente a chamada que estourava o teto
A estimativa do planejamento serve de teto	A estimativa é artefato de planejamento, e o teto tem um piso	A estimativa errou por mais de uma ordem de grandeza
Aprovar na homologação leva à produção	Aprovar integra no ramo de integração; publicar é outra decisão	Separar "a mudança está correta" de "o conjunto deve chegar aos usuários agora"
Similaridade entre embeddings agrupa tickets sobre a mesma dor	Agrupamento de tickets feito por modelo, com o tema explícito	Tickets coloquiais sobre a mesma dor tiveram similaridade menor que pares de temas diferentes

Fonte: elaborado pelos autores (2026), a partir do registro de decisões da PoC.

uma demanda em linguagem natural até a publicação de uma versão do produto, com decisão humana em cada ponto crítico e rastro completo de cada decisão. O fluxo de personalização adaptou a interface por pessoa e recusou, no pior caso, conteúdos maliciosos que contornaram a primeira camada de validação.

Os resultados mostram que a arquitetura funciona de ponta a ponta e, sobretudo, que suas garantias precisam ser verificadas por código para serem garantias. As falhas observadas durante a implementação não invalidam a proposta; ao contrário, cada uma delas foi absorvida pela arquitetura na forma de um mecanismo verificável, o que é precisamente o comportamento que se espera de um sistema projetado para operar com componentes probabilísticos.

6 Conclusão

A IA deixou de ser apenas uma ferramenta de apoio e passou a atuar como agente capaz de planejar, decidir e executar trabalho de software com crescente independência. Este trabalho investigou o que essa mudança exige da arquitetura do software. A análise das cinco dimensões do ecossistema mostrou um padrão consistente: em todas elas, a capacidade técnica já demonstrada contrasta com a ausência de estruturas que a tornem confiável. Para responder a essa lacuna, o trabalho propôs a Arquitetura PASE, implementou uma prova de conceito e a aplicou à evolução de um produto real.

Quanto à primeira questão de pesquisa (QP1), a evidência analisada e a experiência da implementação convergem para a via híbrida. A delegação plena ainda é rara mesmo entre os usuários mais intensos de IA (ANTHROPIC, 2026), e a eficácia de sistemas multiagente depende da adequação entre a coordenação e a tarefa, e não do número de agentes (KIM et al., 2026). Além disso, os casos empresariais mais maduros combinam geração probabilística com execução determinística sob aprovações e auditoria (SAROYA; MACNEILL, 2026). A própria validação reforçou essa conclusão: as falhas mais instrutivas da prova de conceito foram entregas plausíveis que só uma combinação de verificação por código e julgamento humano foi capaz de barrar. O cenário mais provável, e mais desejável, é o de ecossistemas em que a autonomia dos agentes é ampla e, ao mesmo tempo, subordinada a pontos de governança humana.

Quanto à segunda questão (QP2), a PASE responde com um desenho concreto, no qual supervisão, segurança e rastreabilidade não são requisitos não funcionais difusos, mas mecanismos estruturais distribuídos pelas três camadas. A prova de conceito demonstrou que esse desenho é viável. Uma demanda em linguagem natural percorreu as três camadas e chegou à produção com decisão humana em cada ponto crítico. A personalização em tempo de execução, por sua vez, adaptou a interface por usuário sem expor o produto a código arbitrário e recusou, no pior caso, conteúdos maliciosos que contornaram a primeira camada de validação.

6.1 Contribuições

A primeira contribuição deste trabalho é a análise multidimensional do software na era da IA agêntica, que reúne evidências de indústria, desenvolvimento, experiência do usuário, segurança e trabalho e as converte em cinco requisitos explícitos para arquiteturas de software agêntico. Essa conversão permite avaliar propostas futuras por critérios declarados, e não apenas por sua capacidade técnica.

A segunda contribuição é a própria Arquitetura PASE. Ela especifica, em três camadas, como demandas de origens heterogêneas são filtradas com memória do que já foi feito e recusado, e como a decisão humana é posicionada sobre a tarefa, o plano, a entrega e a publicação. Especifica também como a execução é isolada, verificada e revisada de forma independente, e como a personalização em tempo de execução é confinada a áreas com contratos verificáveis. Em relação aos trabalhos mais próximos, a arquitetura não apenas integra frentes antes tratadas isoladamente, mas as subordina a um mesmo regime de autonomia supervisionada.

A terceira contribuição é a prova de conceito e as evidências que ela produziu. A implementação conta com 533 testes automatizados, oito roteiros de demonstração executáveis e registros completos de cada decisão, da proposta bruta à publicação. Seus resultados documentam não apenas o caminho de sucesso, mas as falhas reais encontradas no caminho, como entregas aprovadas que não faziam o que foi pedido e estimativas de custo que erraram por mais de uma ordem de grandeza.

A quarta contribuição é um princípio de projeto que emergiu da validação e atravessa toda a arquitetura: garantias que dependem apenas de instrução ao modelo não são garantias. Em cada falha observada, a correção eficaz consistiu em transferir a garantia para uma verificação por código ou para uma decisão humana informada, com a regra de que o modelo aponta a evidência e o código confere se ela existe. Esse princípio é aplicável a outras arquiteturas que combinem componentes probabilísticos com responsabilidades críticas.

6.2 Limitações

As limitações do estudo começam pela revisão da literatura. Ela foi conduzida como revisão narrativa estruturada, e não como revisão sistemática com protocolo registrado, e parte do corpus é composta por versões preliminares e por relatórios de indústria, escolha justificada pela velocidade do fenômeno estudado, mas que reduz a robustez de algumas evidências. Vários relatórios de indústria, além disso, são produzidos por empresas que comercializam as tecnologias analisadas, e seus casos de uso provêm de clientes ou de empresas investidas. Esses números foram tratados como indicativos de tendência, e não como medições independentes.

A validação também tem alcance limitado. O produto-alvo é de porte reduzido, o número de tarefas executadas é pequeno e os operadores humanos foram os próprios autores, o que impede conclusões estatísticas sobre produtividade, sobre a carga cognitiva da supervisão ou sobre o comportamento da arquitetura em equipes reais. A personalização em tempo de execução foi validada quanto à segurança e ao funcionamento, mas não quanto ao seu efeito sobre a experiência de usuários reais. As métricas de supervisão, por

sua vez, mostraram-se sensíveis à forma de cálculo, como evidenciou a diferença entre a taxa de intervenção na homologação e a taxa de aprovações revertidas.

Por fim, a prova de conceito implementa o núcleo da arquitetura, mas não sua totalidade. A prospecção competitiva, o barramento de eventos entre zonas, os fluxos declarativos de múltiplas etapas, as zonas conversacionais, os subgrafos de dependência, o replanejamento automático de tarefas dependentes e o agendamento periódico dos ciclos de manutenção permaneceram fora do escopo. O isolamento foi implementado por clone e ramo exclusivo, e não por contêiner efêmero por tarefa, e a integração com o GitHub, embora implementada, não foi exercitada na validação aqui relatada. Os modelos de linguagem usados evoluem rapidamente, de modo que resultados quantitativos de custo e de desempenho tendem a mudar com novas versões.

6.3 Trabalhos Futuros

O primeiro desdobramento é completar a implementação dos componentes que permaneceram fora do escopo, em especial o ambiente temporário de visualização para a homologação, a execução em contêineres efêmeros, a integração exercitada com repositórios remotos, as zonas sob demanda conversacionais e o barramento de eventos entre zonas. O ambiente de visualização é particularmente relevante, porque a validação mostrou que parte dos defeitos só se revela quando a pessoa observa o produto em execução.

O segundo desdobramento é uma avaliação em escala e em contexto real. Aplicar a arquitetura a produtos maiores, operados por equipes que não participaram de sua concepção, permitiria medir as métricas de supervisão com amostras significativas e comparar o desempenho com e sem os pontos de decisão humana. Permitiria também avaliar o efeito da personalização sobre usuários reais, inclusive quanto à transparência e à percepção de controle.

O terceiro desdobramento é a investigação dos efeitos sociotécnicos da arquitetura sobre as pessoas que a operam. A literatura indica que a preservação das competências depende de engajamento cognitivo ativo, e não da simples delegação à IA (SHEN; TAMKIN, 2026). Investigar se o pacote de homologação da PASE, com critérios de aceite, evidências conferidas e histórico completo da interação com o modelo, favorece esse engajamento seria uma forma de avaliar se a arquitetura protege, além do produto, a capacidade humana de supervisioná-lo. Com isso, a PASE pretende oferecer um vocabulário e uma estrutura para que a confiabilidade seja tratada como requisito de projeto desde o início, e não como ajuste posterior.

Referências

ABOU ALI, M.; DORNAIKA, F. **Agentic AI**: a comprehensive survey of architectures, applications, and future directions. 2025. ArXiv:2510.25445. Disponível em: <https://arxiv.org/abs/2510.25445>. Acesso em: 12 jan. 2026. Citado 3 vezes nas páginas 19, 24 e 47.

ABSHER, L.; KUPRIYANOVA, O. **State of the agentic AI market report**. [S. l.], 2025. 37 p. Citado 3 vezes nas páginas 1, 5 e 10.

ANTHROPIC. **2026 agentic coding trends report**: how coding agents are reshaping software development. San Francisco, 2026. Disponível em: <https://resources.anthropic.com/hubfs/2026%20Agentic%20Coding%20Trends%20Report.pdf>. Acesso em: 5 fev. 2026. Citado 7 vezes nas páginas 1, 5, 7, 10, 24, 29 e 49.

ARORA, P. et al. **Self-evolving systems**: moving beyond deterministic interfaces to adaptive generative interfaces. [S. l.], 2025. Relatório técnico. Data de publicação não impressa no documento. Disponível em: <https://storage.googleapis.com/gweb-research2023-media/pubtools/1034946.pdf>. Acesso em: 18 nov. 2025. Citado 6 vezes nas páginas 1, 8, 9, 23, 31 e 32.

ASSI, M.; HASSAN, S.; ZOU, Y. LLM-Cure: LLM-based competitor user review analysis for feature enhancement. **ACM Transactions on Software Engineering and Methodology**, New York, v. 35, n. 4, p. 1–25, 2026. Citado na página 15.

BIENIEK, J.; RAHOUTI, M.; VERMA, D. C. **Generative AI in multimodal user interfaces**: trends, challenges, and cross-platform adaptability. 2024. ArXiv:2411.10234. Disponível em: <https://arxiv.org/abs/2411.10234>. Acesso em: 30 abr. 2026. Citado 2 vezes nas páginas 8 e 31.

BURTE, S. AI-powered software development life cycle: from requirements to maintenance. **AI Systems Engineering**, 2025. Disponível em: <https://aise.cultechpub.com/index.php/aise>. Acesso em: 30 abr. 2026. Citado 4 vezes nas páginas 6, 19, 24 e 30.

CAO, Y.; JIANG, P.; XIA, H. Generative and malleable user interfaces with generative and evolving task-driven data model. In: **CHI CONFERENCE ON HUMAN FACTORS IN COMPUTING SYSTEMS, 2025, Yokohama. Proceedings [...]**. New York: ACM, 2025. p. 1–20. Citado na página 16.

CHHABRA, A. et al. Agentic AI security: threats, defenses, evaluation, and open challenges. **IEEE Access**, Piscataway, v. 14, p. 49455–49482, 2026. Citado 4 vezes nas páginas 9, 10, 29 e 33.

CROWDSTRIKE. **2026 global threat report**: year of the evasive adversary. Austin, 2026. 58 p. Disponível em: <https://www.crowdstrike.com/en-us/global-threat-report/>. Acesso em: 18 abr. 2026. Citado na página 9.

GOOGLE CLOUD. **State of AI-assisted software development 2025**. Mountain View, 2025. 142 p. (DORA Report, v. 2025.2). Disponível em: <https://services.google.

com/fh/files/misc/2025_state_of_ai_assisted_software_development.pdf>. Acesso em: 13 abr. 2026. Citado 4 vezes nas páginas 1, 4, 7 e 11.

HE, J.; TREUDE, C.; LO, D. **LLM-based multi-agent systems for software engineering**: literature review, vision and the road ahead. 2025. ArXiv:2404.04834v4. Disponível em: <https://arxiv.org/abs/2404.04834>. Acesso em: 19 out. 2025. Citado na página 7.

HONG, S. et al. MetaGPT: meta programming for a multi-agent collaborative framework. In: **INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS, 12., 2024, Viena. Proceedings** [...]. [S. l.]: ICLR, 2024. Disponível em: <https://openreview.net/forum?id=VtmBAGCN7o>. Acesso em: 17 out. 2026. Citado na página 15.

HOU, X. et al. Large language models for software engineering: a systematic literature review. **ACM Transactions on Software Engineering and Methodology**, New York, v. 33, n. 8, p. 1–79, dez. 2024. Art. 220. Citado 6 vezes nas páginas 3, 6, 9, 21, 22 e 28.

JAMALI, H. et al. AI-powered adaptive learning interfaces: a user experience study in education platforms. **Frontiers in Computer Science**, Lausanne, v. 7, 2025. Art. 1672081. Citado na página 8.

JIMENEZ, C. E. et al. SWE-bench: can language models resolve real-world GitHub issues? In: **INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS, 12., 2024, Viena. Proceedings** [...]. [S. l.]: ICLR, 2024. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 5 nov. 2025. Citado 2 vezes nas páginas 7 e 29.

KIM, Y. et al. **Towards a science of scaling agent systems**. 2026. ArXiv:2512.08296v3. Disponível em: <https://arxiv.org/abs/2512.08296>. Acesso em: 18 jan. 2026. Citado 6 vezes nas páginas 7, 15, 19, 26, 47 e 49.

KRAVCHUK-KIRILYUK, A.; GRACIOLLI, F.; AMIN, N. The modular imperative: rethinking LLMs for maintainable software. In: **ACM SIGPLAN INTERNATIONAL WORKSHOP ON LANGUAGE MODELS AND PROGRAMMING LANGUAGES, 1., 2025, Singapura. Proceedings** [...]. New York: ACM, 2025. p. 1–6. Citado 2 vezes nas páginas 6 e 28.

MUKTI, A. J.; TRISILIA, M. AI-powered adaptive interface: enhancing user experience through real-time personalization in digital platforms. **Procedia Computer Science**, Amsterdam, v. 269, p. 571–580, 2025. Citado na página 8.

SAROYA, M.; MACNEILL, A. **Vista's 2026 outlook**: agentic AI and the future of enterprise software. Austin, 2026. 16 p. Disponível em: <https://www.vistaequitypartners.com/wp-content/uploads/2026/03/Vista-Equity-Partners_AI-Outlook-PDF-Mar-2026.pdf>. Acesso em: 2 abr. 2026. Citado 4 vezes nas páginas 1, 4, 5 e 49.

SHEN, J. H.; TAMKIN, A. **How AI impacts skill formation**. 2026. ArXiv:2601.20245v2. Disponível em: <https://arxiv.org/abs/2601.20245>. Acesso em: 13 abr. 2026. Citado 3 vezes nas páginas 1, 11 e 51.

SPARK. **The enterprise AI transformation journey**: embracing the agentic pivot & beyond. [S. l.], 2026. 32 p. 2026 CIO survey report. Disponível em: <https://wahspark.com/wp-content/uploads/2026/03/SPARK-The-Enterprise-AI-Transformation-Journey.pdf>. Acesso em: 19 abr. 2026. Citado 2 vezes nas páginas 1 e 26.

TAKERNGSAKSIRI, W. et al. Human-in-the-loop software development agents. In: **IEEE/ACM INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING: SOFTWARE ENGINEERING IN PRACTICE, 47., 2025, Ottawa. Proceedings** [...]. Piscataway: IEEE, 2025. Disponível em: <https://arxiv.org/abs/2411.12924>. Acesso em: 17 out. 2026. Citado na página 14.

TAN, L. Y. et al. Artificial intelligence-enabled adaptive learning platforms: a review. **Computers and Education: Artificial Intelligence**, Amsterdam, v. 9, 2025. Art. 100429. Citado 3 vezes nas páginas 8, 9 e 31.

WORLD ECONOMIC FORUM; ACCENTURE. **Organizational transformation in the age of AI**: how organizations maximize AI's potential. Cologny, 2026. 43 p. (Industries in the Intelligent Age). White paper. Disponível em: <https://reports.weforum.org/docs/WEF_Organizational_Transformation_in_the_Age_of_AI_How_Organizations_Maximize_AI%27s_Potential_2026.pdf>. Acesso em: 4 abr. 2026. Citado 3 vezes nas páginas 5, 11 e 25.

ZHANG, Q. et al. A survey on large language models for software engineering. **Science China Information Sciences**, Beijing, v. 69, n. 4, 2026. Art. 141102. Citado 3 vezes nas páginas 6, 21 e 22.

Apêndices

APÊNDICE A – *Prompts* versionados da prova de conceito

A prova de conceito delega a modelos de linguagem apenas decisões específicas, cada uma definida por um *prompt* próprio, mantido como arquivo versionado no repositório, e nunca como texto embutido no código. O Quadro 7 relaciona os quinze *prompts* à camada e ao componente da Arquitetura PASE que cada um implementa.

Quadro 7 – Prompts da prova de conceito e componentes correspondentes

Prompt	**Função**	**Camada e componente**
P0	Extração de intenção a partir de tickets de suporte	Tarefas: captação passiva
P0a	Agrupamento de tickets por tema	Tarefas: captação passiva
P1	Normalização da demanda	Tarefas: *middleware* de normalização
P1b	Avaliação de Zona Dinâmica	Tarefas: Avaliador de Zonas Dinâmicas
P2a	Coerência sistêmica com o contexto do produto	Tarefas: Motor de Filtragem
P2b	Qualificação por impacto, complexidade, confiança e risco	Tarefas: Motor de Filtragem
P3	Geração exploratória de propostas	Tarefas: fluxo da inteligência artificial
P3b	Análise de lacunas entre módulos	Tarefas: fluxo da inteligência artificial
P4	Planejamento técnico e critérios de aceite	Execução: fase de planejamento
P5	Geração e correção de código	Execução: fase de execução
P6	Geração de *payload* de Zona Dinâmica	Execução: UPA
P9	Análise de deriva de contexto dos descartes	Tarefas: Reavaliação Cíclica
P10	Varredura de auditoria do código	Orquestração: Ciclo de Auditoria
P11	Proposta de atualização de um documento de contexto	Orquestração: homologação
P12	Revisão independente da entrega	Execução: revisão independente

Fonte: elaborado pelos autores (2026).

APÊNDICE B – Contrato canônico das Zonas Dinâmicas

O contrato rígido (*Hard Constraint*) que as Zonas Dinâmicas aplicam é um único documento canônico em JSON Schema, validado no servidor antes de persistir cada *payload* e espelhado no cliente, onde ocorre a validação decisiva. Um teste automatizado compara os dois campo a campo. O contrato admite apenas três componentes nativos, rótulos de até quarenta caracteres, no máximo três itens e ligações restritas às rotas internas do produto. Qualquer propriedade não prevista invalida o *payload*.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Contrato das Zonas Dinâmicas",
  "type": "object",
  "additionalProperties": false,
  "required": ["widget", "label"],
  "properties": {
    "widget": { "type": "string", "enum": ["shortcut_list", "summary_card", "alert_banner"] },
    "label": { "type": "string", "minLength": 1, "maxLength": 40 },
    "maxItems": { "type": "integer", "minimum": 1, "maximum": 3 },
    "tone": { "type": "string", "enum": ["neutral", "atencao"] },
    "items": {
      "type": "array",
      "maxItems": 3,
      "items": {
        "type": "object",
        "additionalProperties": false,
        "required": ["label", "href"],
        "properties": {
          "label": { "type": "string", "minLength": 1, "maxLength": 40 },
          "href": {
            "type": "string",
            "pattern": "^/(dashboard|transacoes|relatorios|orcamentos|metas|suporte)$"
          }
        }
      }
    }
  },
  "allOf": [
    {
      "if": { "properties": { "widget": { "enum": ["shortcut_list", "summary_card"] } } },
      "then": { "required": ["maxItems", "items"] }
    },
    {
      "if": { "properties": { "widget": { "const": "alert_banner" } } },
      "then": { "required": ["tone"] }
    }
  ]
```

```
}
```

APÊNDICE C – Reprodução da prova de conceito

A prova de conceito é composta por cinco serviços executados localmente em contêineres: o banco de dados do PASE, o núcleo do PASE, o painel de controle e os dois serviços do FinTrack. O repositório reúne o código dos dois sistemas, os roteiros de demonstração e as suítes de testes. [Indicar aqui como a banca pode acessar o repositório, que atualmente é privado.]

A partir de um ambiente com Docker e com as chaves de acesso ao provedor de modelos configuradas em um arquivo de ambiente local, a sequência de reprodução é a seguinte. O comando `make reset` recria o ambiente do zero e popula o FinTrack e a base inicial do PASE. Os comandos `make test-core` e `make test-client` executam as suítes do núcleo e da interface de programação do FinTrack, e a suíte da interface de usuário é executada no contêiner correspondente. O comando `make demos` executa os oito roteiros de demonstração do Quadro 4, e `make metrics` exporta os arquivos dos quais saem as métricas da Tabela 2. Como os roteiros fazem chamadas reais a modelos de linguagem, sua execução tem custo e pode produzir resultados ligeiramente diferentes a cada vez.

Anexos

Não há anexos. Segundo a NBR 14724, anexos reúnem documentos não elaborados pelos autores que servem de fundamentação ao texto. Todos os materiais de terceiros usados neste trabalho estão citados no texto e listados nas referências, e os materiais complementares elaborados pelos autores estão nos apêndices.